



Bachelor of Information and Communication Technology (BICT)
Honours Degree
CTNT 32051 – Cyber Security Laboratory

Group Assignment – Report

Defensive Security using Wazuh SIEM: Implementation, Log Monitoring, and Threat Detection

Group Members:

CT/2021/012 - Weerakkodi W.A.S.k

CT/2021/005 - Vidarshana K G

CT/2021/024 - Jayalath J.M.S.P

CT/2021/039 - Sasikala B R S

CT/2021/029 - Athapaththu T.I

Table of Content

1. Introduction

- 1.1 Background
- 1.2 What is a SIEM System?
- 1.3 What is Wazuh?
- 1.4 Purpose and Scope of This Assignment

2. System Architecture and Setup

- 2.1 Wazuh Architecture Overview
- 2.2 Components of Wazuh
- 2.3 System Architecture Diagram
- 2.4 Data Flow and Communication
- 2.5 Lab Environment Specification

3. Installation Procedure

- 3.1 Wazuh Server Installation on Ubuntu 22.04 LTS
- 3.2 Wazuh Agent Deployment

4. Configuration Details

- 4.1 Overview
- 4.2 System Log Configuration
- 4.3 Authentication Log Configuration
- 4.4 Application Log Configuration
- 4.5 Restarting the Agent to Apply Configuration
- 4.6 Configuration for Proper Communication Between Components
- 4.7 File Integrity Monitoring Configuration

5. Log Monitoring Process

- 5.1 Overview of Wazuh Dashboard
- 5.2 System Logs Monitoring
- 5.3 Authentication Logs Monitoring

5.4 Application Logs Monitoring

5.5 File Integrity Monitoring

5.6 Demonstration of Simulated Events

6. Screenshots of Implementation

6.1 Wazuh Dashboard

6.2 Security Event panel

6.3 Alert detail view

6.4 Net Runtime alert

6.5 MITRE ATT&CK panel

6.6 System Logs Monitoring View

6.7 File Integrity Monitoring Dashboard

7. Security Event Detection and Analysis

7.1 Overview of Detected Security Events

7.2 Summary of Triggered Alerts

8. Analysis of Detected Events

8.1 Detailed Alert Investigation

8.2 Root Cause and Attack Methodology

8.3 Threat Impact Assessment

8.4 How Wazuh Identifies and Responds to Threats

9. Conclusion

9.1 Summary of Findings

9.2 Reflection on Wazuh's Effectiveness

9.3 Possible Improvements

9.4 Future Work

10. References

1.Introduction

1.1 Background

Given the fast-paced environment of cyber-attacks that organizations encounter in the current digital age, there is a continuous increase in the number of cybersecurity threats being witnessed. Cybersecurity experts are always faced with the challenge of finding ways to detect cyber threats and mitigate their impacts. Given that the volume and complexity of cybersecurity events have grown in proportion to IT infrastructure, a need has emerged for organizations to develop automated systems that can intelligently monitor security incidents.

Cybersecurity defense is an array of technologies and methodologies used to safeguard information systems against malicious attacks, damage, or unauthorized use. Of all the essential components that make up cybersecurity defense mechanisms, the Security Information and Event Management (SIEM) system ranks as one of the most significant tools. SIEM technology offers a comprehensive solution that integrates with an organization's network architecture and monitors its cybersecurity performance in real-time [1].

1.2 What is a SIEM System?

Security Information and Event Management (SIEM) is one of the cybersecurity solutions designed to monitor, collect, and analyze data security-related information from different systems and devices within the network. Using this type of software, organizations can detect security threats, investigate security incidents, and ensure their information security.

As the name suggests, a SIEM system includes both SIM and SEM solutions.

The first part of this solution is Security Information Management (SIM), which is primarily concerned with log collection and storage from different sources over a period. After any security event, this data can be further investigated and used for reporting or auditing purposes.

The other component of the solution is Security Event Management (SEM). Unlike SIM, it allows continuous monitoring of security-related events within an organization's systems. The activities are continuously analyzed according to certain criteria and compared with existing rules. In case of violation, the user receives a report about the event immediately.

Data collected by the SIEM system comes from various sources within the organization. For example:

- Operating systems
- Firewalls
- Intrusion Detection Systems (IDS)
- Web servers
- Databases
- Network devices
- Cloud services

All this data comes together in one central system, where it becomes easy to analyze. This makes it possible for administrators to have an overview of the security situation in the network and solve any problem quickly.

Furthermore, the SIEM system helps organizations fulfill security and compliance obligations since it automatically logs and generates reports. Besides, it enhances incident response through quick detection and resolution of security threats [1].

Therefore, the SIEM system is an essential component in contemporary cybersecurity due to its centralized monitoring capabilities, real-time threat detection, and enhanced security of computer systems and networks. The SIEM solution used in this project for monitoring and security management is Wazuh [18].

1.3 What is Wazuh?

Wazuh is a free, open-source unified security platform that delivers comprehensive SIEM and Extended Detection and Response (XDR) capabilities. Originally formed from the OSSEC open-source intrusion detection system, Wazuh has evolved into a full-featured security monitoring platform actively maintained by Wazuh Inc. and a large open-source community. It is used by organizations ranging from small businesses to government agencies worldwide [2].

Wazuh is built on a modular, scalable architecture and integrates with the OpenSearch stack (formerly Elasticsearch and Kibana) to provide powerful data indexing, search, and visualization.

Key Security Features:

- Log Data Analysis [5]- Harvests and analyzes log data generated by endpoints and network devices using an advanced rule-based engine that utilizes over 3,000 detection rules covering various operating systems, applications, and attack methods.
- Intrusion Detection [2]- Identifies attack signatures, rootkits, and malware using signature-based detection and anomaly-based analysis of system calls and log patterns.
- File Integrity Monitoring (FIM) [19]- Monitors specific files and directories for unauthorized access, changes, or deletion. Wazuh generates cryptographic hashes of selected files and directories and alerts when anything changes.
- Vulnerability Detection [2]- Cross references installed software packages on monitored endpoints with National Vulnerability Database (NVD) and Common Vulnerabilities and Exposures (CVE) to detect known vulnerabilities.
- Security Configuration Assessment (SCA) [7]- Detects misconfigured settings in systems based on CIS benchmarks to highlight potential vulnerabilities that can be leveraged by the attackers.
- Active Response [11]- Allows automated mitigation of threats using pre-defined scripts when certain security conditions occur, such as blacklisting the IP address of an attacker through the firewall.
- Cloud Security Monitoring - Monitors cloud environments including Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP).
- Regulatory Compliance Reporting [6]- Pre-built reports for regulatory compliance like PCI DSS, HIPAA, GDPR, NIST 800-53, MITRE ATT&CK, and TSC SOC2.

The open-source nature of Wazuh makes this tool highly suitable for the academic environment where it becomes possible to analyze the detection rules, understand the alerting process, and modify the application [2].

1.4 Purpose and Scope of This Assignment

Introduction:

The objective of this project was to give practical and direct experience in installing and using a SIEM tool in a lab setting. The report outlines the entire procedure carried out by the team, which entails preparing the environment, installing all the Wazuh components, configuring the logging and file integrity monitoring, simulating security events, monitoring using the Wazuh Dashboard, and analyzing the generated alerts [3].

The scope of work for this project includes:

- Installation of Wazuh in a lab environment consisting of VMs running Ubuntu 22.04 LTS [3].
- Installation and testing of all the components of Wazuh – Indexer, Manager/Server, Agent, and Dashboard [3].
- Configuring log sources from various sources such as authentication, system, and package manager logs [5].
- Configuring File Integrity Monitoring for important directories on the system [19].
- Monitoring simulated security incidents to test the detection capabilities of the SIEM solution [10].

2. System Architecture and Setup

This chapter explains the architecture of the Wazuh SIEM solution, and the particular virtual lab environment used for this project. Knowledge of architecture is essential before implementation since each constituent part performs unique and crucial functions in the process of collecting, analyzing, storing, and visualizing security events [3].

2.1 Wazuh Architecture Overview

In the case of the Wazuh solution, the client-server model is used with four distinct components that interact with one another and help provide full security monitoring. Before setting up Wazuh, it is critical to be familiar with the function of each component since all these components create a pipeline of functions from endpoint data gathering up to visualization [3].

2.2 Components of Wazuh

Wazuh Indexer

The Wazuh Indexer is a largely scalable, full-text search and analytics engine and analytics machine grounded on OpenSearch. It serves as the data continuity subcaste of the Wazuh platform, responsible for storing and recording all security cautions and archived event data generated by the Wazuh Manager. The Hand exposes a REST API that allows both the Wazuh Garcon (to write cautions) and the Wazuh Dashboard (to read and query cautions) to interact with the stored data. In product surroundings, the Hand can be stationed as a multi-node cluster to handle high volumes of event data with redundancy [3].

Wazuh Server (Manager)

The Wazuh Server is the central analysis and management component of the platform. When event data is received from connected agents, the Server performs the following functions in sequence: decoding of raw log messages into structured fields using decoder files; application of a rule set against the decoded data to identify security events; generation of alerts when rules are matched; forwarding of alerts and archived events to the Wazuh Indexer; and management of agent registration, configuration distribution, and communication through the Wazuh Registration Service and Cluster Daemon [3].

Wazuh Dashboard

The Wazuh Dashboard is a web-based graphical user interface built on top of OpenSearch Dashboards [2]. It provides security analysts with real-time and historical visibility into security events, agent status, file integrity changes, vulnerability reports, and compliance dashboards. The Dashboard communicates directly with the Wazuh Indexer to retrieve alert data and with the Wazuh Server REST API to provide agent management functionality. It is accessed through a standard web browser over HTTPS on port 443.

Wazuh Agent

The Wazuh Agent is a small piece of software which is installed on the monitored endpoint like servers, desktops, and VMs. This software constantly gathers information that relates to security from the monitored endpoint. This information includes OS logs, filesystem modifications, process list, network connections and, in case of Windows-based systems, registry modification. All gathered information is then encrypted with TLS and transmitted over port 1514 (TCP/UDP) to the Wazuh server. Wazuh agent works as a service which impacts performance minimally [4].

2.3 System Architecture Diagram

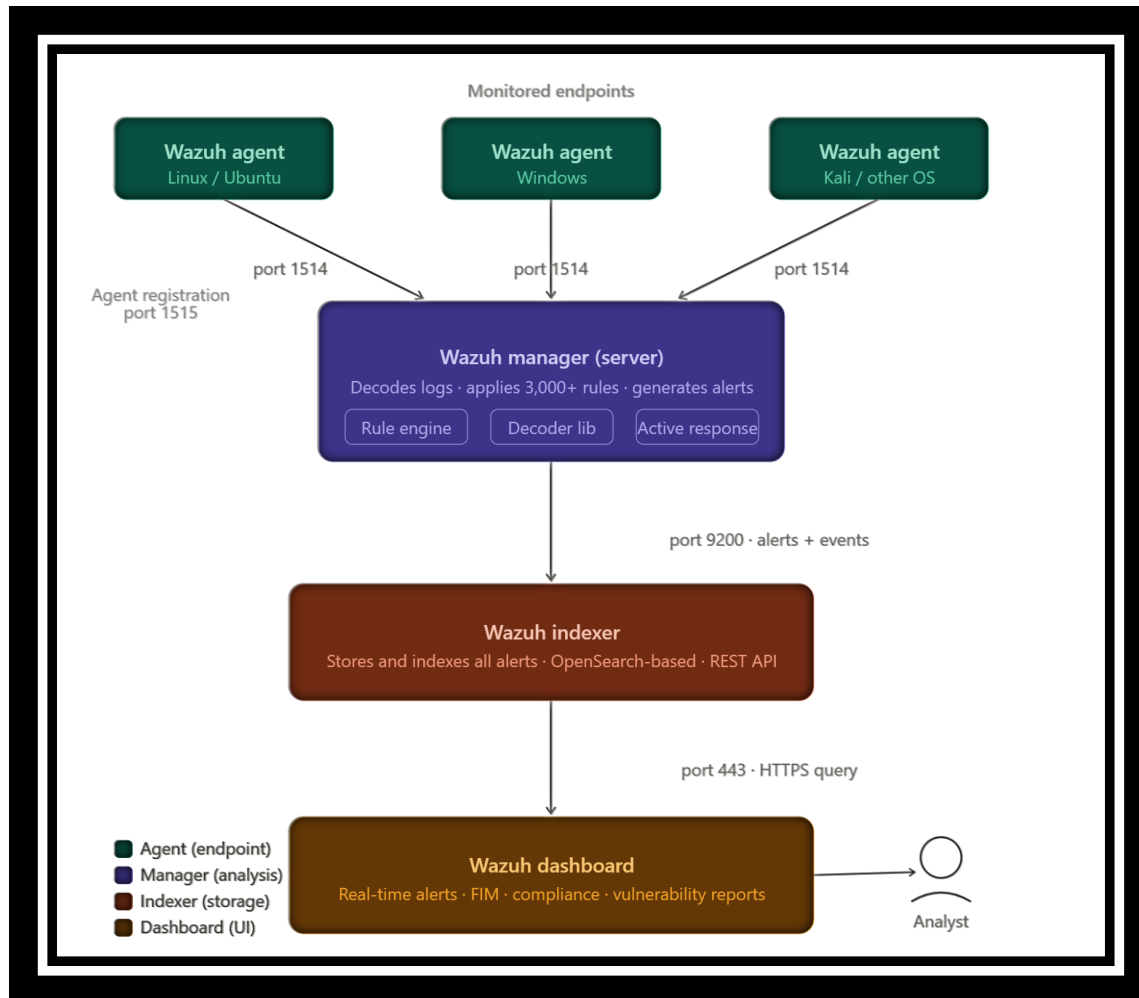


Figure 2.1 - Wazuh SIEM System Architecture

2.4 Data Flow and Communication

The following table describes the end-to-end flow of security data through the Wazuh platform, from initial collection at the endpoint to presentation in the analyst dashboard[3]:

Stage	Description
1 Collection	Wazuh Agents collect log data, file system events, and security telemetry from monitored endpoints and forward encrypted data to the Wazuh Server on port 1514 [3].
2 Decoding	The Wazuh Server applies decoder rules to parse raw log messages into structured fields (e.g., source IP, username, event type, timestamp) [3].
3 Rule Matching	Decoded events are evaluated against the Wazuh rule engine (3,000+ built-in rules). Matched rules generate security alerts with a severity level (0–15) [3].
4 Indexing	Generated alerts and archived events are forwarded from the Server to the Wazuh Indexer on port 9200 for storage and retrieval [3].
5 Visualization	The Wazuh Dashboard queries the Indexer over HTTPS (port 443) and presents alerts, trends, and compliance data to the security analyst through the web interface [2],[3].
6Response (Optional)	Active response scripts can be triggered on the agent by the Server to automatically mitigate detected threats, such as blocking a source IP address [11].

The six-stage pipeline described above reflects the standard Wazuh data flow architecture.

2.5 Lab Environment Specifications

Below is the virtualized lab environment setup used for this assignment [3]. The two virtual machines, which represent one Wazuh Server and another monitored machine with the Wazuh Agent installed, were set up using one physical host:

Wazuh Server Virtual Machine

Parameter	Specification
Hypervisor	Oracle VirtualBox
Operating System	Ubuntu 22.04 LTS (64-bit Server)
Server IP Address	192.168.8.155 / 24 (network interface: enp0s3)
RAM Allocated	4 GB
Storage	25 GB
CPU Cores	2 virtual CPU cores
Network	Bridged Adapter
Wazuh Version	4.7.5 — all-in-one installation (Indexer + Manager + Dashboard)
Services Running	wazuh-indexer · wazuh-manager · wazuh-dashboard
Dashboard URL	https://192.168.8.155:443
Admin Login	Username: admin Password: auto-generated during installation

Note on IP Address Differences: This lab was conducted across two physical machines at different times. The Wazuh agent was installed and tested on two separate Windows PCs during the session:

- Primary machine — Agent IP: 172.20.107.212 — used for authentication log monitoring, system log monitoring, application log monitoring and simulated attack scenarios
- Secondary machine — Agent IP: 192.168.8.146 — used for file integrity monitoring testing

As a result, some screenshots in this report show different IP addresses depending on which machine was active at the time of capture. Both machines connected to the same Wazuh server at 192.168.8.155 and all monitoring data was collected through the same dashboard.

Wazuh Agent Endpoint

Parameter	Agent 002 – Windows 11
Agent Name	MyPcWazuagent
Operating System	Windows 11 Home single Language 10.0.26200.8246
IP Address	172.20.107.212
Wazuh Agent Version	v4.7.5
Installation Method	msiexec (.msi via PowerShell)
Manager Address	192.168.8.155
Dashboard Status	Active

Network Topology Summary

Machine	Role	IP Address	Operating System
ubuntu-VirtualBox	Wazuh Server (Indexer + Manager + Dashboard)	192.168.8.155	Ubuntu 22.04 LTS
MyPcWazuagent	Wazuh Agent (monitored endpoint)	192.168.8.146	Windows 11 Pro

In this lab architecture, a realistic implementation of the SIEM system for an enterprise is shown, where the Wazuh server performs centralized monitoring of several endpoints, which run on various operating systems. This example of the Windows-based endpoint shows the ability of the Wazuh server to perform cross-platform monitoring.

3. Step – by – Step Installation Procedure

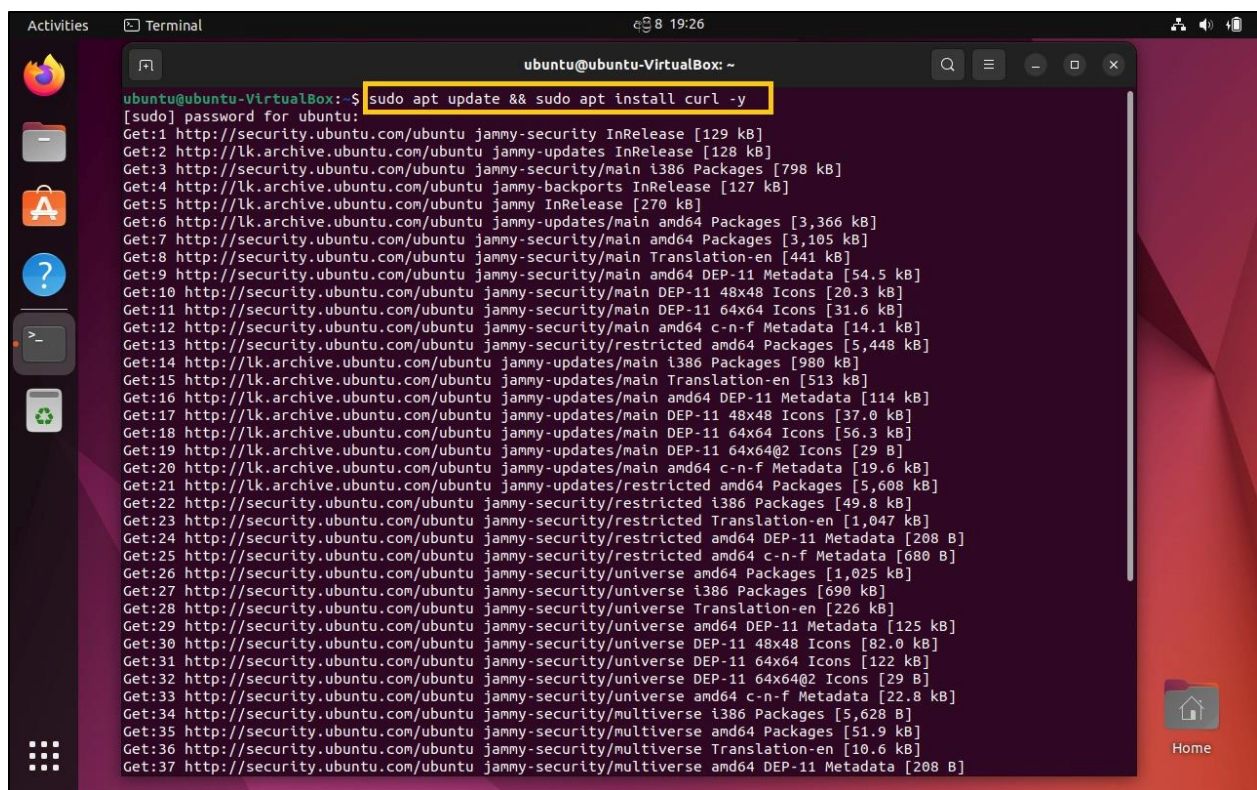
3.1 Wazuh Server Installation on Ubuntu 22.04 LTS

Wazuh server installation was done using an official installation script. Using the script, all necessary components such as the server, indexer, and dashboard were installed automatically [3].

Step 1 – Update System Packages and Install curl

The first step involved updating all the packages of the Ubuntu system and installing the curl tool. This was necessary for the downloading process of the Wazuh installation script in the next step. The following command was issued

```
sudo apt update && sudo apt install curl -y
```



```
ubuntu@ubuntu-VirtualBox: ~  
ubuntu@ubuntu-VirtualBox:~$ sudo apt update && sudo apt install curl -y  
[sudo] password for ubuntu:  
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]  
Get:2 http://lk.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]  
Get:3 http://security.ubuntu.com/ubuntu jammy-security/main i386 Packages [798 kB]  
Get:4 http://lk.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]  
Get:5 http://lk.archive.ubuntu.com/ubuntu jammy InRelease [270 kB]  
Get:6 http://lk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3,366 kB]  
Get:7 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3,105 kB]  
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [441 kB]  
Get:9 http://security.ubuntu.com/ubuntu jammy-security/main amd64 DEP-11 Metadata [54.5 kB]  
Get:10 http://security.ubuntu.com/ubuntu jammy-security/main DEP-11 48x48 Icons [20.3 kB]  
Get:11 http://security.ubuntu.com/ubuntu jammy-security/main DEP-11 64x64 Icons [31.6 kB]  
Get:12 http://security.ubuntu.com/ubuntu jammy-security/main amd64 c-n-f Metadata [14.1 kB]  
Get:13 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [5,448 kB]  
Get:14 http://lk.archive.ubuntu.com/ubuntu jammy-updates/main i386 Packages [980 kB]  
Get:15 http://lk.archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [513 kB]  
Get:16 http://lk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 DEP-11 Metadata [114 kB]  
Get:17 http://lk.archive.ubuntu.com/ubuntu jammy-updates/main DEP-11 48x48 Icons [37.0 kB]  
Get:18 http://lk.archive.ubuntu.com/ubuntu jammy-updates/main DEP-11 64x64 Icons [56.3 kB]  
Get:19 http://lk.archive.ubuntu.com/ubuntu jammy-updates/main DEP-11 64x64@2 Icons [29 B]  
Get:20 http://lk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.6 kB]  
Get:21 http://lk.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [5,608 kB]  
Get:22 http://security.ubuntu.com/ubuntu jammy-security/restricted i386 Packages [49.8 kB]  
Get:23 http://security.ubuntu.com/ubuntu jammy-security/restricted Translation-en [1,047 kB]  
Get:24 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 DEP-11 Metadata [208 B]  
Get:25 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 c-n-f Metadata [680 B]  
Get:26 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1,025 kB]  
Get:27 http://security.ubuntu.com/ubuntu jammy-security/universe i386 Packages [690 kB]  
Get:28 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en [226 kB]  
Get:29 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 DEP-11 Metadata [125 kB]  
Get:30 http://security.ubuntu.com/ubuntu jammy-security/universe DEP-11 48x48 Icons [82.0 kB]  
Get:31 http://security.ubuntu.com/ubuntu jammy-security/universe DEP-11 64x64 Icons [122 kB]  
Get:32 http://security.ubuntu.com/ubuntu jammy-security/universe DEP-11 64x64@2 Icons [29 B]  
Get:33 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 c-n-f Metadata [22.8 kB]  
Get:34 http://security.ubuntu.com/ubuntu jammy-security/multiverse i386 Packages [5,628 B]  
Get:35 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 Packages [51.9 kB]  
Get:36 http://security.ubuntu.com/ubuntu jammy-security/multiverse Translation-en [10.6 kB]  
Get:37 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 DEP-11 Metadata [208 B]
```

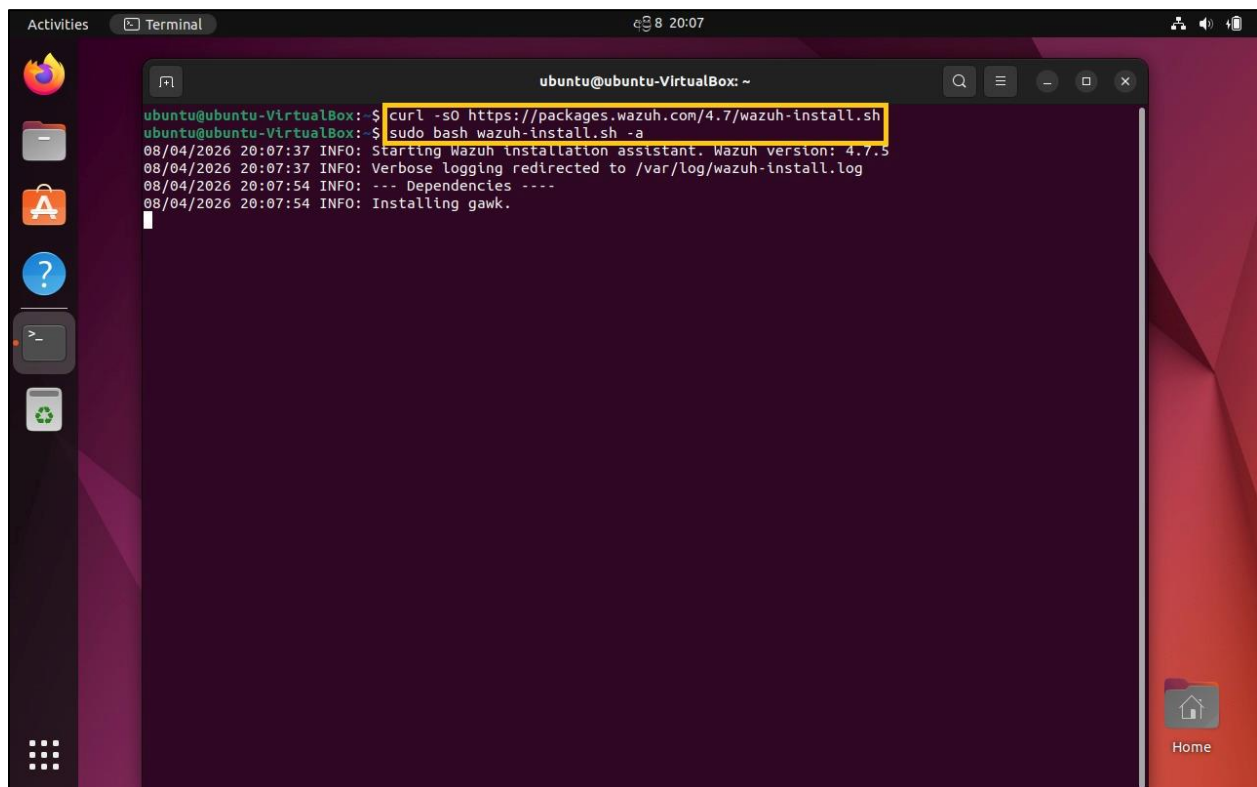
Figure 3.1 - Ubuntu Update Process

Purpose:

- Update package lists
- Install latest updates
- Prepare the system for Wazuh installation

Step 2 – Download and Execute the Wazuh Installation Script

The automated installation script of the Wazuh software (version 4.7) was obtained from the Wazuh Package Repository and ran directly by using the -a flag [3]. The -a flag stands for 'all-in-one' and tells the script to install all three parts of the Wazuh server components, which include Wazuh Indexer, Wazuh Manager, and Wazuh Dashboard, on the system automatically.

A screenshot of a terminal window on a Linux desktop. The terminal shows the execution of a script to install Wazuh. The user runs 'curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh' and then 'sudo bash wazuh-install.sh -a'. The output shows the script starting the Wazuh installation assistant, logging to /var/log/wazuh-install.log, checking dependencies, and installing gawk. The desktop background is a red and purple geometric pattern. The terminal window title is 'ubuntu@ubuntu-VirtualBox: ~'.

```
ubuntu@ubuntu-VirtualBox: ~  
ubuntu@ubuntu-VirtualBox:~$ curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh  
ubuntu@ubuntu-VirtualBox:~$ sudo bash wazuh-install.sh -a  
08/04/2026 20:07:37 INFO: Starting Wazuh installation assistant. Wazuh version: 4.7.5  
08/04/2026 20:07:37 INFO: Verbose logging redirected to /var/log/wazuh-install.log  
08/04/2026 20:07:54 INFO: --- Dependencies ----  
08/04/2026 20:07:54 INFO: Installing gawk.
```

Figure 3.2 - Wazuh Package Installation

```
# Download the Wazuh installation assistant
```

```
curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
```

```
# Execute the all-in-one installation
```

```
sudo bash wazuh-install.sh -a
```

Purpose:

- Install Wazuh Server
- Install Wazuh Indexer
- Install Wazuh Dashboard
- Configure default services

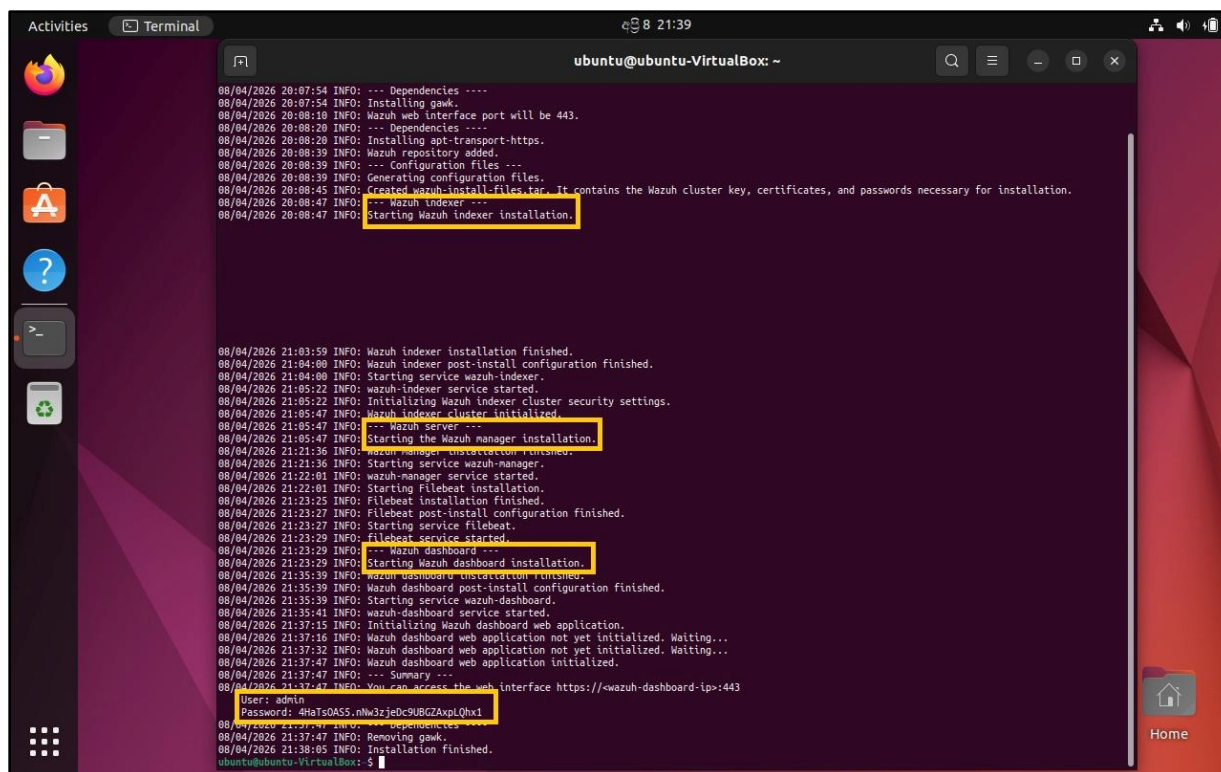
Step 3 – Installation Complete: Retrieve Admin Credentials

Once the installation was completed successfully, the following default username and password for accessing the Wazuh Dashboard were automatically generated by the system.

The credentials generated are as follows:

- Username: admin
- Password: (Automatically generated password appearing on terminal screen)

The credentials thus generated were safely stored and used for logging into the Wazuh dashboard interface.

A terminal window titled 'ubuntu@ubuntu-VirtualBox: ~' showing the output of a Wazuh installation script. The logs include timestamps and status messages for various components. Three sections are highlighted with yellow boxes: 'Starting Wazuh Indexer installation.', 'Starting the Wazuh manager installation.', and 'Starting Wazuh dashboard installation.'. At the bottom, a summary section provides the user 'admin' and a password, along with the web interface URL 'https://wazuh-dashboard-ip:443'.

```
08/04/2026 20:07:54 INFO: --- Dependencies ---
08/04/2026 20:07:54 INFO: Installing gawk.
08/04/2026 20:08:10 INFO: Wazuh web interface port will be 443.
08/04/2026 20:08:20 INFO: --- Dependencies ---
08/04/2026 20:08:20 INFO: Installing apt-transport-https.
08/04/2026 20:08:39 INFO: Wazuh repository added.
08/04/2026 20:08:39 INFO: --- Configuration files ---
08/04/2026 20:08:39 INFO: Generating configuration files.
08/04/2026 20:08:45 INFO: Created wazuh-install-files.tar. It contains the Wazuh cluster key, certificates, and passwords necessary for installation.
08/04/2026 20:08:47 INFO: --- Wazuh Indexer ---
08/04/2026 20:08:47 INFO: Starting Wazuh Indexer installation.

08/04/2026 21:03:59 INFO: Wazuh indexer installation finished.
08/04/2026 21:04:00 INFO: Wazuh indexer post-install configuration finished.
08/04/2026 21:04:00 INFO: Starting service wazuh-indexer.
08/04/2026 21:05:22 INFO: wazuh-indexer service started.
08/04/2026 21:05:22 INFO: Initializing Wazuh indexer cluster security settings.
08/04/2026 21:05:47 INFO: --- Wazuh Server ---
08/04/2026 21:05:47 INFO: Starting the Wazuh manager installation.
08/04/2026 21:21:36 INFO: Wazuh manager installation finished.
08/04/2026 21:21:36 INFO: Starting service wazuh-manager.
08/04/2026 21:22:01 INFO: wazuh-manager service started.
08/04/2026 21:22:01 INFO: Starting filebeat installation.
08/04/2026 21:23:25 INFO: Filebeat installation finished.
08/04/2026 21:23:27 INFO: Filebeat post-install configuration finished.
08/04/2026 21:23:27 INFO: Starting service filebeat.
08/04/2026 21:23:29 INFO: filebeat service started.
08/04/2026 21:23:29 INFO: --- Wazuh dashboard ---
08/04/2026 21:23:29 INFO: Starting Wazuh dashboard installation.
08/04/2026 21:35:39 INFO: Wazuh dashboard installation finished.
08/04/2026 21:35:39 INFO: Wazuh dashboard post-install configuration finished.
08/04/2026 21:35:39 INFO: Starting service wazuh-dashboard.
08/04/2026 21:35:41 INFO: wazuh-dashboard service started.
08/04/2026 21:37:15 INFO: Initializing Wazuh dashboard web application.
08/04/2026 21:37:16 INFO: Wazuh dashboard web application not yet initialized. Waiting...
08/04/2026 21:37:32 INFO: Wazuh dashboard web application not yet initialized. Waiting...
08/04/2026 21:37:47 INFO: Wazuh dashboard web application initialized.
08/04/2026 21:37:47 INFO: --- Summary ---
08/04/2026 21:37:47 INFO: You can access the web interface https://wazuh-dashboard-ip:443
User: admin
Password: 4heTs0ASS_nNw3rjeDc9UBGZxplQhx1
08/04/2026 21:37:57 INFO: --- Dependencies ---
08/04/2026 21:37:57 INFO: Removing gawk.
08/04/2026 21:38:05 INFO: Installation finished.
ubuntu@ubuntu-VirtualBox: $
```

Figure 3.3 - Installation Success and Credential

Figure 3.3 shows the terminal output after executing the installation script. As observed from the highlighted sections in the figure, the installation of the three components was done successfully:

- Installation of Wazuh Indexer completed → Wazuh indexer service started → Cluster setup.
- Installation of Wazuh Manager (server) completed → Wazuh manager service started → Filebeat started.
- Installation of Wazuh Dashboard completed → Wazuh dashboard service started → Web interface initialized.

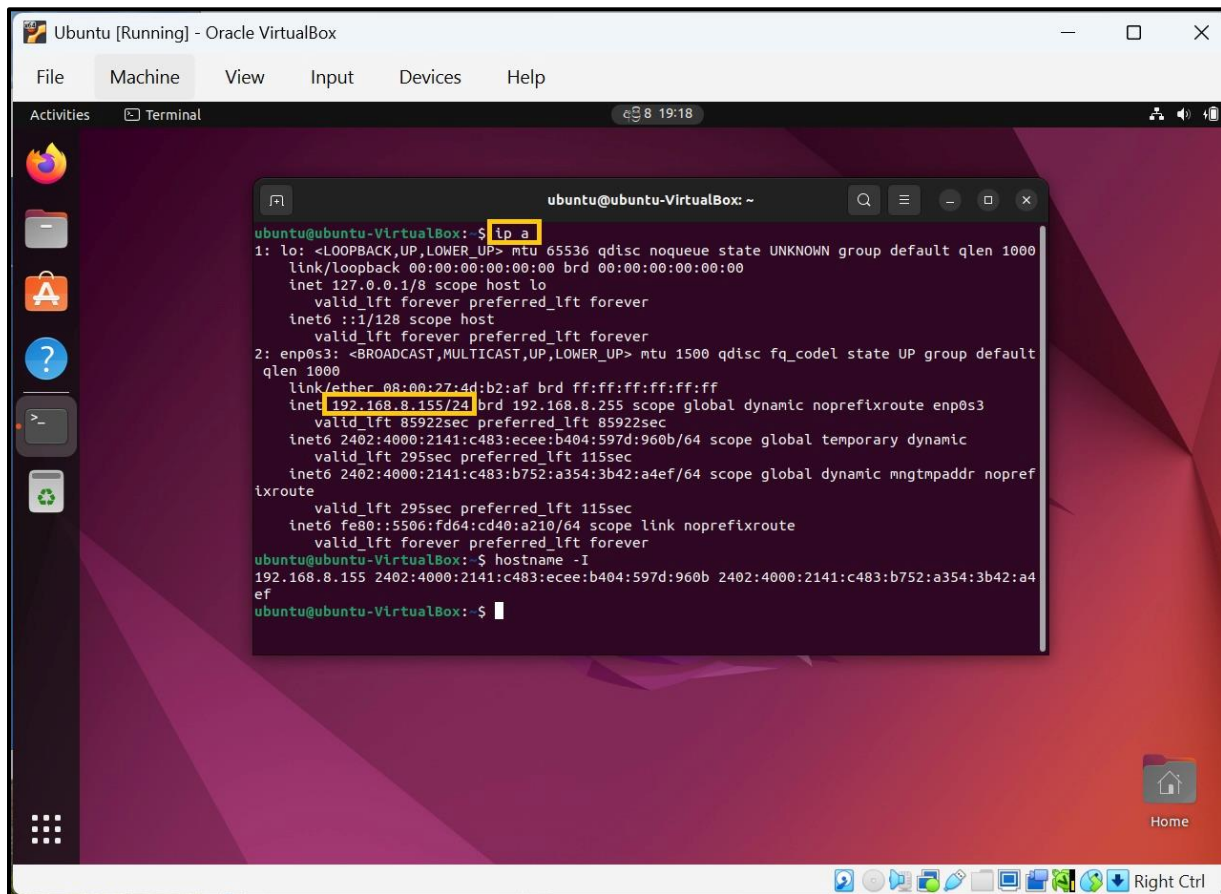
Step 4 – Identify the Wazuh Server IP Address

Before accessing the Dashboard or deploying agents, the IP address of the Ubuntu Server VM was confirmed using the **ip a** and **hostname -I** commands. This IP address is used in the browser URL to access the Dashboard and is configured in all agents as the **WAZUH_MANAGER** address.

Purpose:

- Identify server IP address
- Enable web-based dashboard access

```
ip a
hostname -I
```



```
ubuntu@ubuntu-VirtualBox: ~  
ubuntu@ubuntu-VirtualBox: $ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host  
        valid_lft forever preferred_lft forever  
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default  
    qlen 1000  
    link/ether 08:00:27:4d:b2:af brd ff:ff:ff:ff:ff:ff  
    inet 192.168.8.155/24 brd 192.168.8.255 scope global dynamic noprefixroute enp0s3  
        valid_lft 85922sec preferred_lft 85922sec  
    inet6 2402:4000:2141:c483:ecee:b404:597d:960b/64 scope global temporary dynamic  
        valid_lft 295sec preferred_lft 115sec  
    inet6 2402:4000:2141:c483:b752:a354:3b42:a4ef/64 scope global dynamic mngtmpaddr nopref  
ixroute  
        valid_lft 295sec preferred_lft 115sec  
    inet6 fe80::5506:fd64:cd40:a210/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
ubuntu@ubuntu-VirtualBox: $ hostname -I  
192.168.8.155 2402:4000:2141:c483:ecee:b404:597d:960b 2402:4000:2141:c483:b752:a354:3b42:a4  
ef  
ubuntu@ubuntu-VirtualBox: $
```

Figure 3.4 – Ip address display

Figure 3.4 confirms the network configuration of the Ubuntu VM. The `ip a` command output shows interface `enp0s3` is assigned IP address `192.168.8.155/24` with broadcast `192.168.8.255` — highlighted in the screenshot. The `hostname -I` command also returns `192.168.8.155` as the primary address. This IP was used for all subsequent Wazuh configuration steps.

Step 5 – Access Wazuh Dashboard Using Server IP Address

Access to the Wazuh dashboard was gained by typing the IP address of the server in the web browser. A security alert popped up on the screen because of the SSL certificate. This is normal in a lab setting, so we proceeded to click ‘Proceed’ and then logged into Wazuh via the link below:

```
URL:   https://192.168.8.155
Username: admin
Password:
```

The previously generated username and password were used to log in successfully to the Wazuh Dashboard interface.

Purpose:

- Access Wazuh Dashboard
- Verify successful installation
- Begin system monitoring

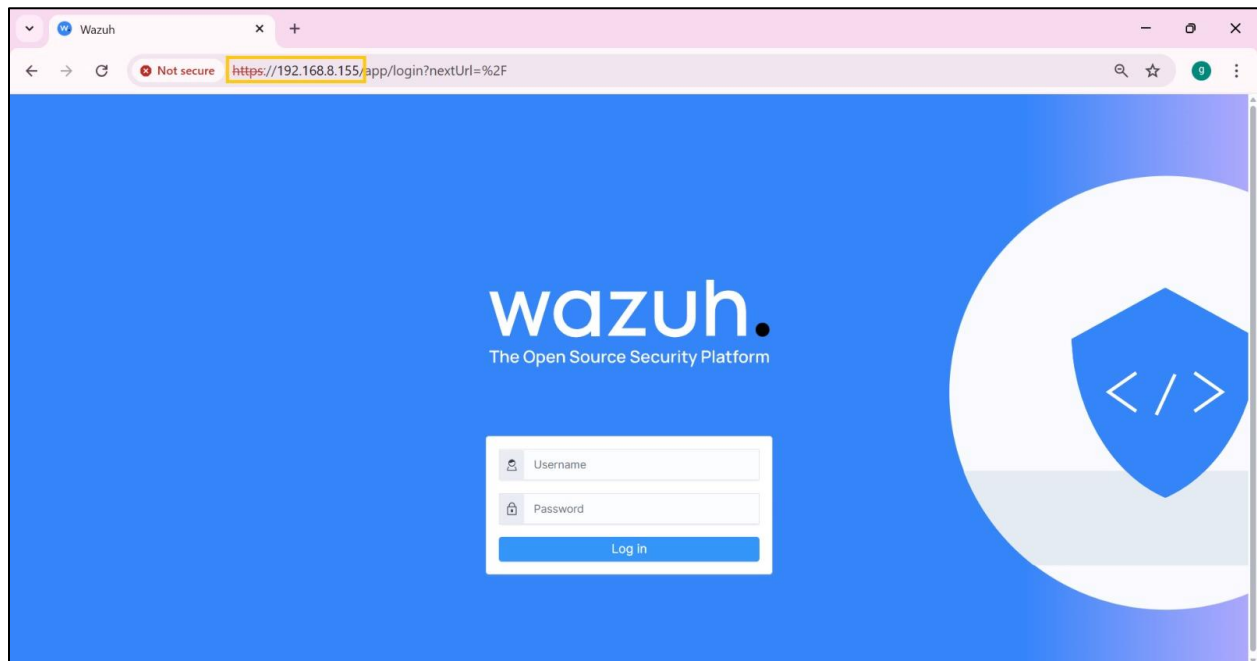


Figure 3.5 – Wazuh Dashboard Login

The Wazuh Dashboard login page can be seen opening on Figure 3.5, with Chrome being used. The address bar verifies that the dashboard is indeed available through <https://192.168.8.155>, with the 'Not secure' tag appearing because of the self-signed certificate, which is expected during lab configurations. The admin's login details from Step 3 were used to sign in.

Wazuh Dashboard accessible at <https://192.168.8.155>. Login successful with admin credentials.

3.2 Wazuh Agent Deployment

The Wazuh agent was installed on a Windows system through the use of the MSI installer file. It was configured using the server IP address [4].

Step 6 – Deploy New Agent via Wazuh Dashboard

The deployment of the new Wazuh Agent was performed using the built-in 'Deploy new agent' wizard provided by the Wazuh Dashboard. This utility provides automatic generation of the necessary commands depending on the operating system, architecture, and server IP [4]. The 'Deploy new agent' wizard can be accessed via the 'Agents' section.

Purpose:

- To register a new monitoring node
- To prepare a machine for log management

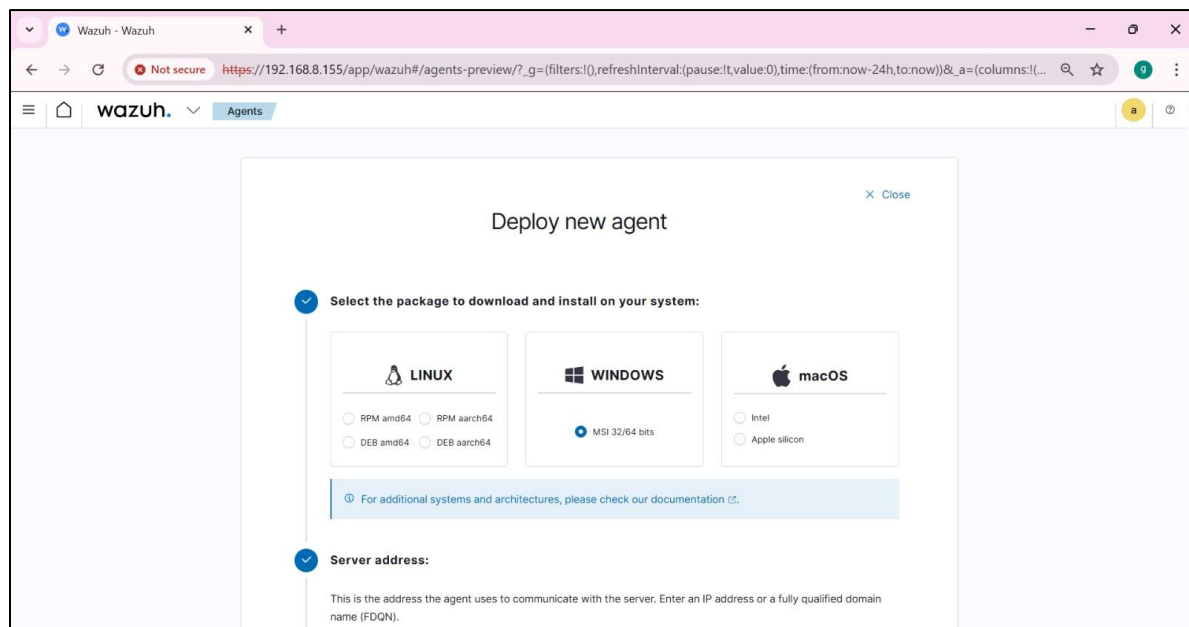


Figure 3.6 – Agent Creation

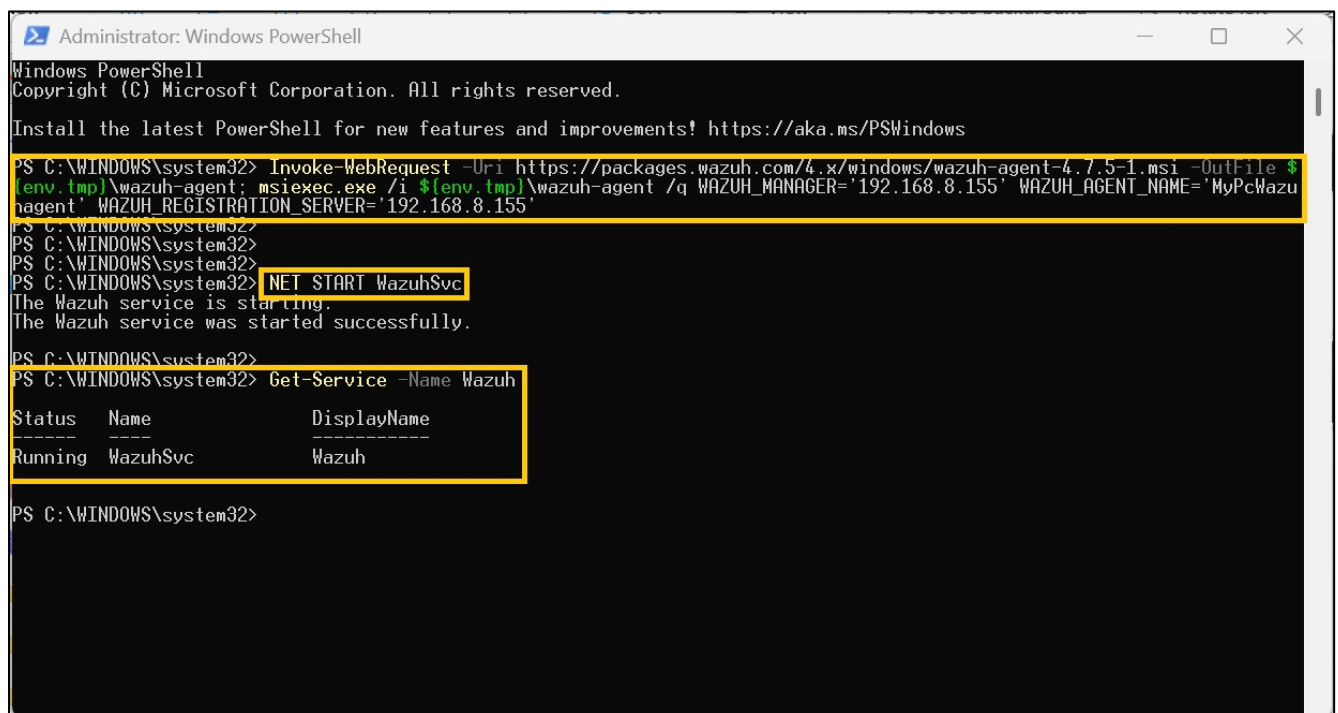
Figure 3.6 below represents the deployment wizard for deploying agents into the Wazuh dashboard. The wizard gives you the chance to select among three operating systems, namely Linux, Windows, and macOS, with different architecture options for each operating system. In the case of installing the Windows 11 agent, the option 'Windows MSI 32/64 bits' is chosen. Afterward, the wizard asks you for the server address (192.168.8.155) and an agent name before generating the commands.

Step 7 – Install Wazuh Agent on Windows Using PowerShell

The agent creation step followed by the execution of the installation command through the Windows PowerShell terminal resulted in the successful installation of the Wazuh Agent on the Windows environment [4].

Following steps were performed:

1. Open Windows PowerShell as Administrator
2. Copy agent installation command from Wazuh Dashboard
3. Execute the installation command



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.7.5-1.msi -OutFile $(env:tmp)\wazuh-agent.msi
PS C:\WINDOWS\system32> msixexec.exe /i $(env:tmp)\wazuh-agent.msi /q WAZUH_MANAGER='192.168.8.155' WAZUH_AGENT_NAME='MyPcWazu
agent' WAZUH_REGISTRATION_SERVER='192.168.8.155'
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> NET START WazuhSvc
The Wazuh service is starting.
The Wazuh service was started successfully.
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Get-Service -Name Wazuh

Status  Name      DisplayName
-----
Running WazuhSvc  Wazuh

PS C:\WINDOWS\system32>
```

Figure 3.7 - PowerShell Agent Installation

The screenshot provided in Figure 3.7 depicts that the Wazuh Agent was successfully installed and started on the Windows 11 system. The key outputs from the PowerShell terminal are as follows:

- The Invoke-WebRequest command fetches the wazuh-agent-4.7.5-1.msi file from the official Wazuh site and saves it to the local computer's temp directory.
- msixexec.exe installs the wazuh-agent-4.7.5-1.msi file silently, with WAZUH_MANAGER and WAZUH_AGENT_NAME predefined arguments, hence automatically registering the agent at 192.168.8.155 under the name 'MyPcWazuhagent'.
- The NET START WazuhSvc command starts the Wazuh service successfully, which is indicated by the terminal response stating 'The Wazuh service was started successfully.'
- The Get-Service command checks the status of the Wazuh service, displaying its status as 'Running'.

Conclusion:

The Wazuh Agent was installed successfully and registered to the Wazuh Server.

Wazuh Agent (WazuhSvc) installed and running on Windows 11.

Agent name: MyPcWaz

Step 8 – Verify Agent in Wazuh Dashboard

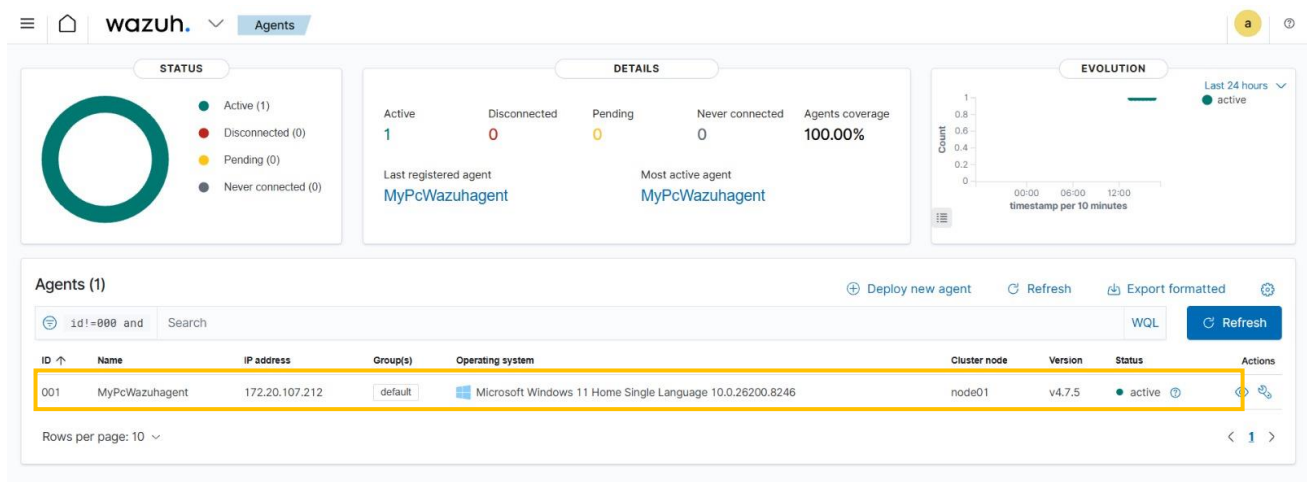
After completing agent installation on the Windows 11 machine, the Wazuh Dashboard Agents panel was refreshed to corroborate that agent had successfully connected and registered with the Wazuh Manager. The Agents runner at [https:// 192.168.8.155/ app/ wazuh/ agents- exercise](https://192.168.8.155/app/wazuh/agents-exercise) was accessed to confirm their status.

Status displayed

- Agent Status Active
- Connection Successful

Purpose

- Confirm successful agent enrollment
- corroborate system monitoring functionality



4. Configuration details

4.1 Overview

Following the successful installation of the Wazuh server and implementation of the Windows 11 agent, it was now time to configure the sources of logs as well as the monitoring options to allow for detection of security events. Configuration of all agents was done through the configuration file of Wazuh Agent as follows [3]:

C:\Program Files (x86)\ossec-agent\ossec.conf

Once changes were made in the file, it was important to restart the service as below:

```
Restart-Service -Name WazuhSvc
```

4.2 System Log Configuration

Windows system logs are collected through the Windows Event Log. The following block was added to ossec.conf to collect System channel events [5]:

```
<localfile>
  <location>System</location>
  <log_format>eventchannel</log_format>
</localfile>
```

Purpose: Captures hardware failures, service crashes, driver errors, and system-level warnings generated by the Windows OS kernel and core services [9].

4.3 Authentication Log Configuration

Authentication events are among the most security-critical logs in any environment. The following block was configured to collect Windows Security channel events, which include all login attempts, account lockouts, and privilege use [15],[16]:

```
<localfile>
  <location>Security</location>
  <log_format>eventchannel</log_format>
  <query>Event/System[EventID != 5145 and EventID != 5156 and EventID !=
5447 and
  EventID != 4656 and EventID != 4658 and EventID != 4663 and EventID !=
4660 and
  EventID != 4670 and EventID != 4690 and EventID != 4703 and EventID !=
4907 and
  EventID != 5152 and EventID != 5157]</query>
</localfile>
```


Objective: Helps to identify instances of login failures (Event ID 4625), successful logins (Event ID 4624), account lockouts (Event ID 4740), and privilege escalations (Event ID 4672). These are the key signs of brute force attacks and unauthorized access [15],[16].

4.4 Application Log Configuration

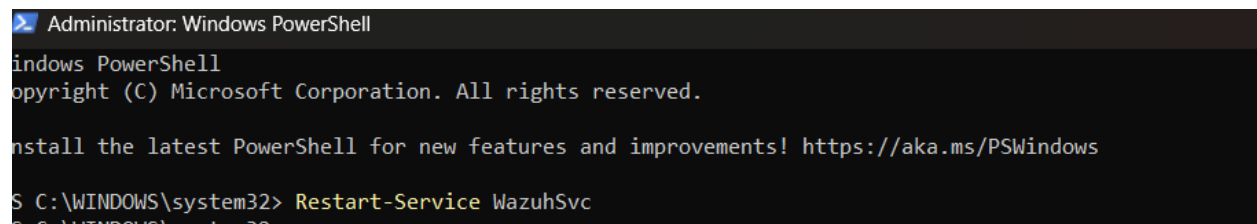
Application logs record events from installed software and services. The following block is collected from the Windows Application channel:

```
<!-- Log analysis -->
<localfile>
  <location>Application</location>
  <log_format>eventchannel</log_format>
</localfile>
```

Purpose: Detects application crashes, service failures, and software-level errors that may indicate exploitation attempts or unstable system behavior caused by malware [5].

4.5 Restarting the Agent to Apply Configuration

After all log source configurations were added to ossec.conf, the Wazuh Agent service was restarted to load the updated settings:



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Restart-Service WazuhSvc
```

4.6 Configuration for Proper Communication Between Components

The system setup was done in a manner that would make sure that there is effective communication between the Wazuh server and the agent machine. The process included setting up the Wazuh server and also configuring the Windows agent in order for there to be successful communication between the two machines.

Installation and Initialization of Wazuh Server Components

The Wazuh SIEM application was set up on an Ubuntu Server operating system. As the process of setting up was going on, the system itself automatically set up the necessary components for the security operations and log handling.

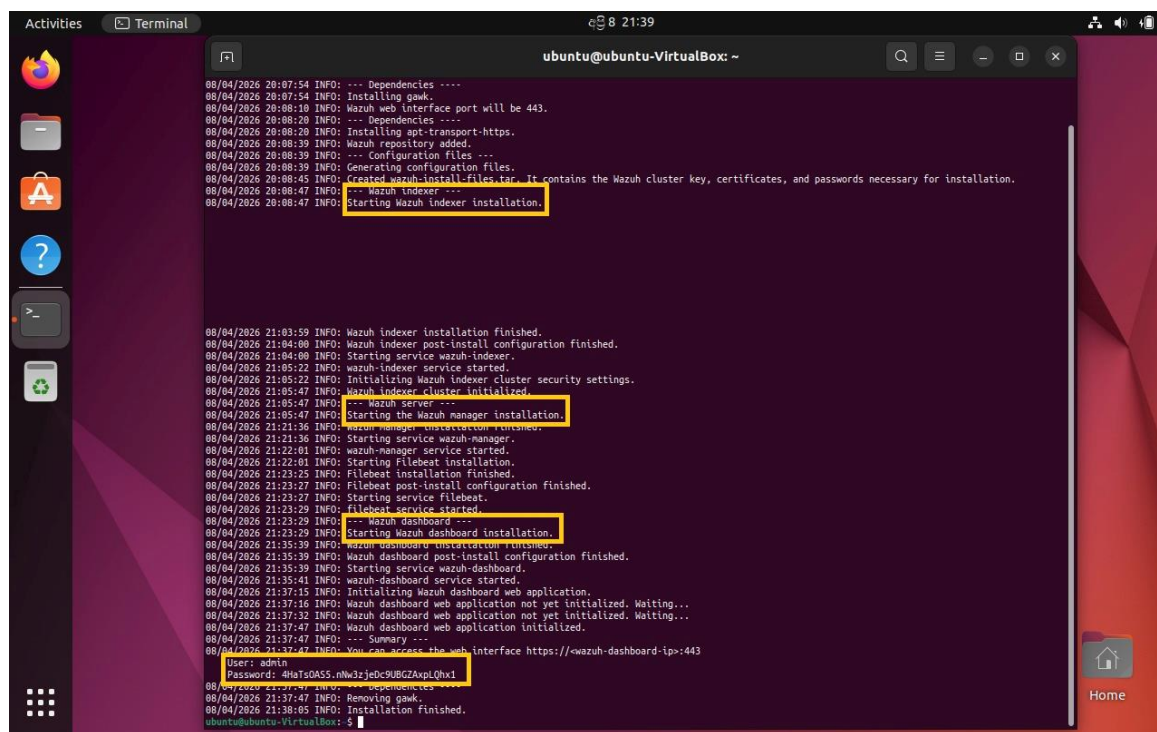
The following key components have been correctly set up and initialized:

- Wazuh Indexer
- Wazuh Manager
- Wazuh Dashboard

The process went well without any problems, and all of the services were started successfully. Once the installation was completed, the system provided with the default administrator credentials to gain access to the web management console.

The dashboard service has been configured to use port 443 in order to provide secure access via the HTTPS protocol.

With the installation and initialization processes done successfully, it can be concluded that the server environment has been set up correctly to handle the agent connections.



```
08/04/2026 20:07:54 INFO: --- Dependencies ---
08/04/2026 20:07:54 INFO: Installing gawk.
08/04/2026 20:08:10 INFO: Wazuh web interface port will be 443.
08/04/2026 20:08:20 INFO: --- Dependencies ---
08/04/2026 20:08:20 INFO: Installing apt-transport-https.
08/04/2026 20:08:39 INFO: Wazuh repository added.
08/04/2026 20:08:39 INFO: --- Configuration files ---
08/04/2026 20:08:39 INFO: Generating configuration files.
08/04/2026 20:08:45 INFO: Created wazuh-install-files.tar. It contains the Wazuh cluster key, certificates, and passwords necessary for installation.
08/04/2026 20:08:47 INFO: --- Wazuh indexer ---
08/04/2026 20:08:47 INFO: Starting Wazuh indexer installation.

08/04/2026 21:03:59 INFO: Wazuh indexer installation finished.
08/04/2026 21:04:00 INFO: Wazuh indexer post-install configuration finished.
08/04/2026 21:04:00 INFO: Starting service wazuh-indexer.
08/04/2026 21:05:22 INFO: wazuh-indexer service started.
08/04/2026 21:05:22 INFO: Initializing Wazuh indexer cluster security settings.
08/04/2026 21:05:47 INFO: Wazuh indexer cluster initialized.
08/04/2026 21:05:47 INFO: --- Wazuh server ---
08/04/2026 21:05:47 INFO: Starting the Wazuh manager installation.
08/04/2026 21:11:36 INFO: wazuh-manager post-install configuration finished.
08/04/2026 21:11:36 INFO: Starting service wazuh-manager.
08/04/2026 21:22:01 INFO: wazuh-manager service started.
08/04/2026 21:22:01 INFO: Starting Filebeat installation.
08/04/2026 21:23:25 INFO: Filebeat installation finished.
08/04/2026 21:23:27 INFO: Filebeat post-install configuration finished.
08/04/2026 21:23:27 INFO: Starting service filebeat.
08/04/2026 21:23:29 INFO: filebeat service started.
08/04/2026 21:23:29 INFO: --- Wazuh dashboard ---
08/04/2026 21:23:29 INFO: Starting Wazuh dashboard installation.
08/04/2026 21:35:39 INFO: Wazuh dashboard installation finished.
08/04/2026 21:35:39 INFO: Wazuh dashboard post-install configuration finished.
08/04/2026 21:35:39 INFO: Starting service wazuh-dashboard.
08/04/2026 21:35:41 INFO: wazuh-dashboard service started.
08/04/2026 21:37:15 INFO: Initializing Wazuh dashboard web application.
08/04/2026 21:37:16 INFO: Wazuh dashboard web application not yet initialized. Waiting...
08/04/2026 21:37:32 INFO: Wazuh dashboard web application not yet initialized. Waiting...
08/04/2026 21:37:47 INFO: Wazuh dashboard web application initialized.
08/04/2026 21:37:47 INFO: --- Summary ---
08/04/2026 21:37:47 INFO: You can access the web interface https://wazuh-dashboard-ip:443
User: admin
Password: 4H4Ts0K55.nM3zjE0c9UBGZAplQhX1
08/04/2026 21:37:47 INFO: Removing gawk.
08/04/2026 21:38:05 INFO: Installation finished.
ubuntu@ubuntu-VirtualBox: $
```

Figure: Successful installation of Wazuh server components and generation of dashboard login credentials

Configuration and Installation of the Windows Agent

The Windows system was used to configure the system as an agent to gather information about the logs of the system and pass them to the server for analysis. This configuration was done using Windows PowerShell by gaining administrative access.

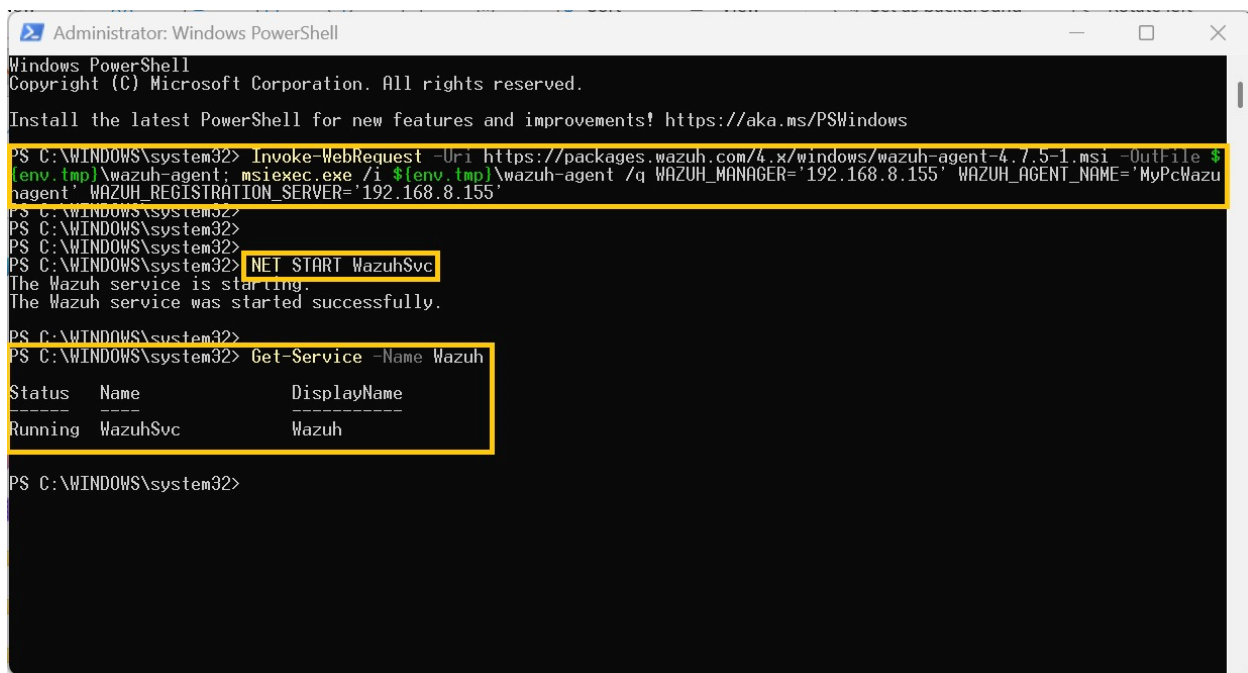
In configuring the system, the IP address of the server was set to create a connection between the agent and the server. Also, the system name was defined so that the system could be easily identified in the monitoring environment.

The following were the configuration parameters set in the process of installing the agent:

IP address of the server: 192.168.8.155

Name of the agent: MyPCwazuagent

The installation was successful, and the service of Wazuh agent was up and running on the system.



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.7.5-1.msi -OutFile $(env:tmp)\wazuh-agent; msixexec.exe /i $(env:tmp)\wazuh-agent /q WAZUH_MANAGER='192.168.8.155' WAZUH_AGENT_NAME='MyPcWazuagent' WAZUH_REGISTRATION_SERVER='192.168.8.155'
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> NET START WazuhSvc
The Wazuh service is starting.
The Wazuh service was started successfully.
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Get-Service -Name Wazuh

Status  Name      DisplayName
-----
Running WazuhSvc  Wazuh

PS C:\WINDOWS\system32>
```

Figure: Installation and configuration of Wazuh agent on Windows system using PowerShell

Agent Communication Check in Ubuntu Server

In the below picture, there is the verification process for the communication check between the Wazuh server, and the agent connected with it. The command run in Ubuntu server gives the list of registered agents and their connection status.

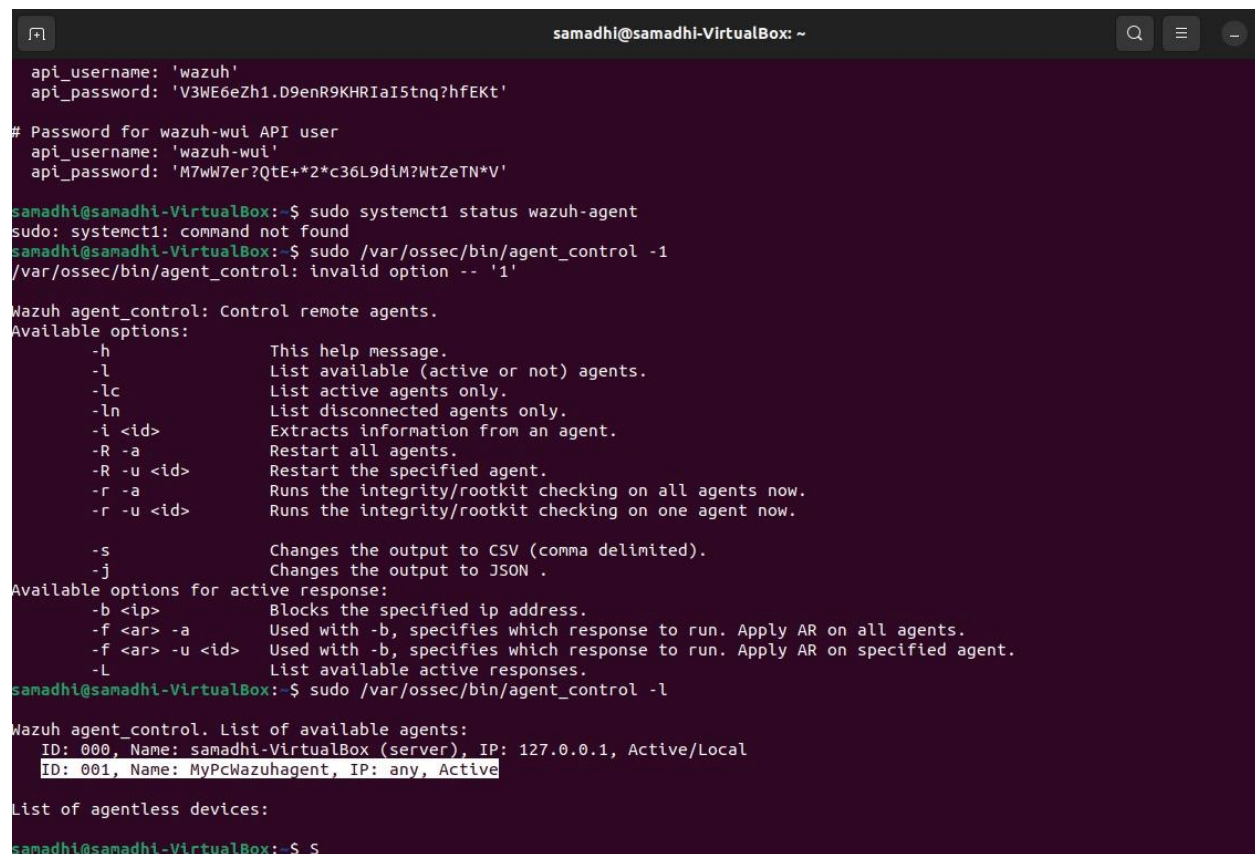
Command used:

```
sudo /var/ossec/bin/agent_control -l
```

Output indicates that there is the agent registered with its unique ID and name with the communication status as “active,” confirming the communication check between the server and the agent.

Objective of this step:

- To verify the agent registration
- To confirm the communication
- To verify the agent connection status

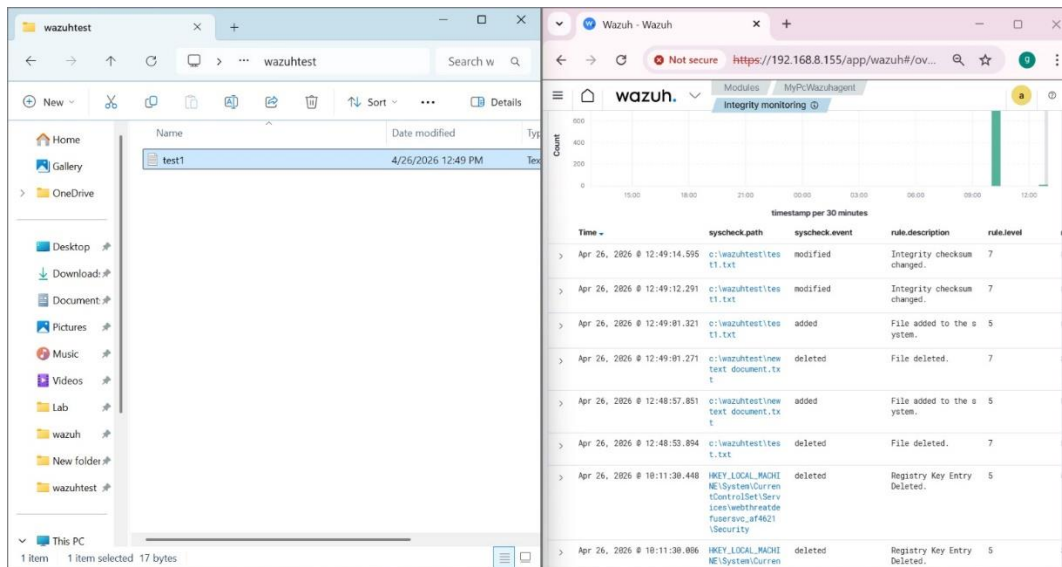


```
samadhi@samadhi-VirtualBox: ~  
api_username: 'wazuh'  
api_password: 'V3WE6eZh1.D9enR9KHRIaI5tnq?hfEkt'  
# Password for wazuh-wui API user  
api_username: 'wazuh-wui'  
api_password: 'M7wW7er?QtE+*2*c36L9diM?WtZeTN*V'  
samadhi@samadhi-VirtualBox:~$ sudo systemctl status wazuh-agent  
sudo: systemctl: command not found  
samadhi@samadhi-VirtualBox:~$ sudo /var/ossec/bin/agent_control -l  
/var/ossec/bin/agent_control: invalid option -- 'l'  
  
Wazuh agent_control: Control remote agents.  
Available options:  
-h          This help message.  
-l          List available (active or not) agents.  
-lc         List active agents only.  
-ln         List disconnected agents only.  
-i <id>     Extracts information from an agent.  
-R -a       Restart all agents.  
-R -u <id>  Restart the specified agent.  
-r -a       Runs the integrity/rootkit checking on all agents now.  
-r -u <id>  Runs the integrity/rootkit checking on one agent now.  
  
-s          Changes the output to CSV (comma delimited).  
-j          Changes the output to JSON .  
Available options for active response:  
-b <ip>     Blocks the specified ip address.  
-f <ar> -a   Used with -b, specifies which response to run. Apply AR on all agents.  
-f <ar> -u <id> Used with -b, specifies which response to run. Apply AR on specified agent.  
-L          List available active responses.  
samadhi@samadhi-VirtualBox:~$ sudo /var/ossec/bin/agent_control -l  
  
Wazuh agent_control. List of available agents:  
ID: 000, Name: samadhi-VirtualBox (server), IP: 127.0.0.1, Active/Local  
ID: 001, Name: MyPcWazuhagent, IP: any, Active  
  
List of agentless devices:  
samadhi@samadhi-VirtualBox:~$ S
```

Figure: Verification of active communication between Wazuh server and registered agent.

4.7 File Integrity Monitoring Configuration

In order to detect file-level changes on the Windows machine, the syscheck module was activated from within ossec.conf [19]. This was done using a test directory C:\wazuhtest that was made specifically for that.



```
ossec.conf
File Edit View
<windows_apps>./shared/win_applications_rcl.txt</windows_apps>
<windows_malware>./shared/win_malware_rcl.txt</windows_malware>
</rootcheck>

<!-- Security Configuration Assessment -->
<sca>
  <enabled>yes</enabled>
  <scan_on_start>yes</scan_on_start>
  <interval>12h</interval>
  <skip_nfs>yes</skip_nfs>
</sca>

<!-- File integrity monitoring -->
<syscheck>
  <disabled>no</disabled>

  <!-- Monitor Documents folder-->
  <directories check_all="yes" realtime="yes">C:\wazuhtest</directories>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

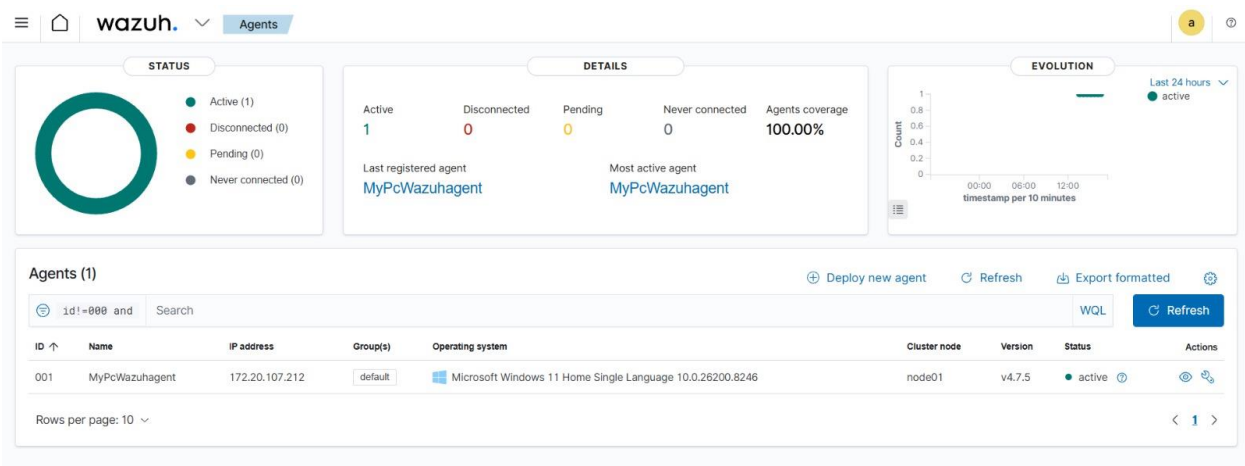
  <!-- Default files to be monitored. -->
  <directories recursion_level="0" restrict="regedit.exe$|system.ini$|win.ini$">%WINDIR%</directories>

  <directories recursion_level="0" restrict="at.exe$|attrib.exe$|caccls.exe$|cmd.exe$|eventcreate.exe$|ftp.exe$|lsass.exe$|net.exe$|net1.exe$|netsh.exe$|
```

Purpose: To detect any unauthorized files addition, modification, deletion, or change in their integrity checksum within the directory in question [19].

5. Log Monitoring Process

5.1 Overview of Wazuh Dashboard



Overview of Wazuh Dashboard

The Wazuh Dashboard was accessed directly through a web browser using the server's IP address. It gave us a single place to see everything (active agents, security incidents, and system activity) without jumping between tools.

The Agents panel confirmed one registered agent for this session:

001 — MyPcWazuagent · Microsoft Windows 11 Home Single Language 10.0.26200.8246 · IP 172.20.107.212 · Status: Active

Agent coverage sat at 100%, meaning every registered device was online and communicated with the server. MyPcWazuagent was both the last registered and the most active agent throughout the session.

The main dashboard sections we worked with were:

- Security Events [5] - for viewing and filtering all triggered alerts
- Agents Overview [2] - for confirming connected devices and their status
- MITRE ATT&CK Dashboard [6] - for mapping detected events to known attack techniques

From this single interface, we could track exactly what was happening on the monitored Windows machine in real time without needing direct access to the device itself.

5.2 System Logs Monitoring

>	Apr 24, 2026 @ 15:43:18.312		Wazuh agent started.	3	503
~	Apr 24, 2026 @ 15:42:57.757	T1562.001	Defense Evasion	3	506

Table	JSON	Rule
	@timestamp	2026-04-24T10:12:57.757Z
	_id	ex36vp0B0wVM0eCHKpUV
	agent.id	001
 	agent.ip	172.20.107.212
	agent.name	MyPcWazuHagent
	data.extra_data	MyPcWazuHagent->any
	decoder.name	ossec
	decoder.parent	ossec
	full_log	ossec: Agent stopped: 'MyPcWazuHagent->any'.
	id	1777025577.2753079
	input.type	log
	location	wazuh-remoted
	manager.name	samadhi-VirtualBox

There's a lot happening inside a Windows machine that never makes it to the screen.

- Services quietly stopped
- Hardware throwing warnings
- The OS firing off alerts in the background

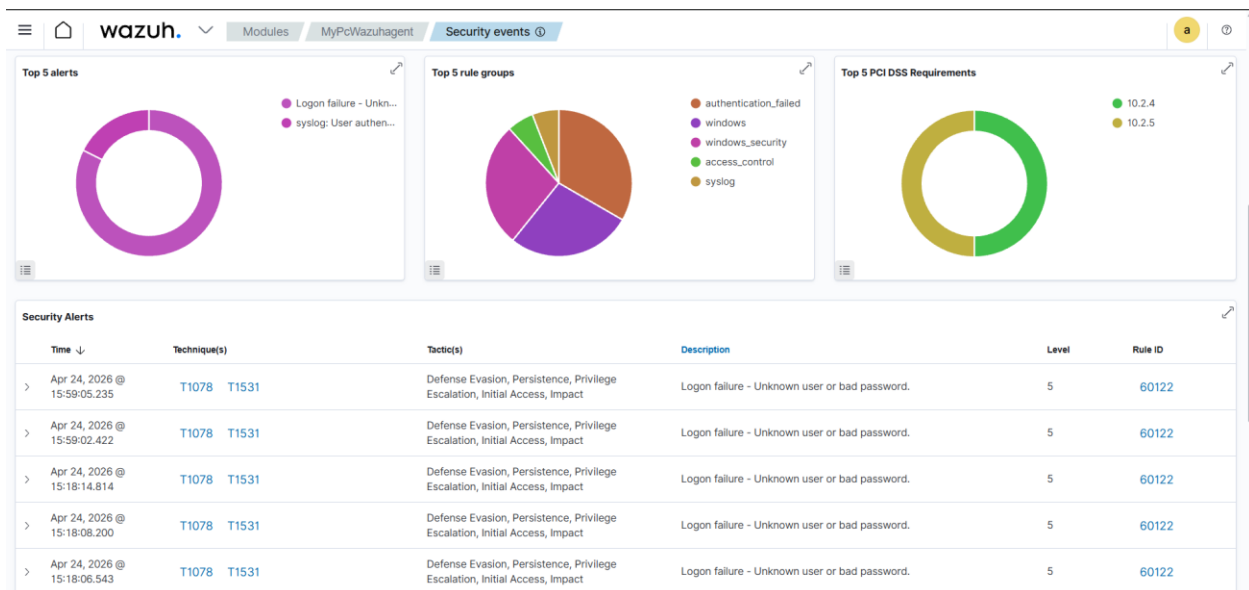
To catch all of that, we made one change we went into `ossec.conf` on the Windows agent and configured it to pull from the System Event Log channel.

That was it. From that moment, logs started arriving at the Wazuh server automatically. No manual steps. No missed events.

On the dashboard, everything organized itself under the Windows rules group.

Suddenly, the machine wasn't a black box anymore. We could see exactly what was happening at the system level in real time, across every device we were monitoring.

5.3 Authentication Logs Monitoring



This is where the failed login attempts became impossible to ignore.

The dashboard told the story briefly the Top 5 Alerts chart was almost entirely dominated by one thing: logon failures; Unknown users; Bad passwords; Repeated attempts hitting the same machine [14].

Drilling into the Security Alerts table confirmed it [10].

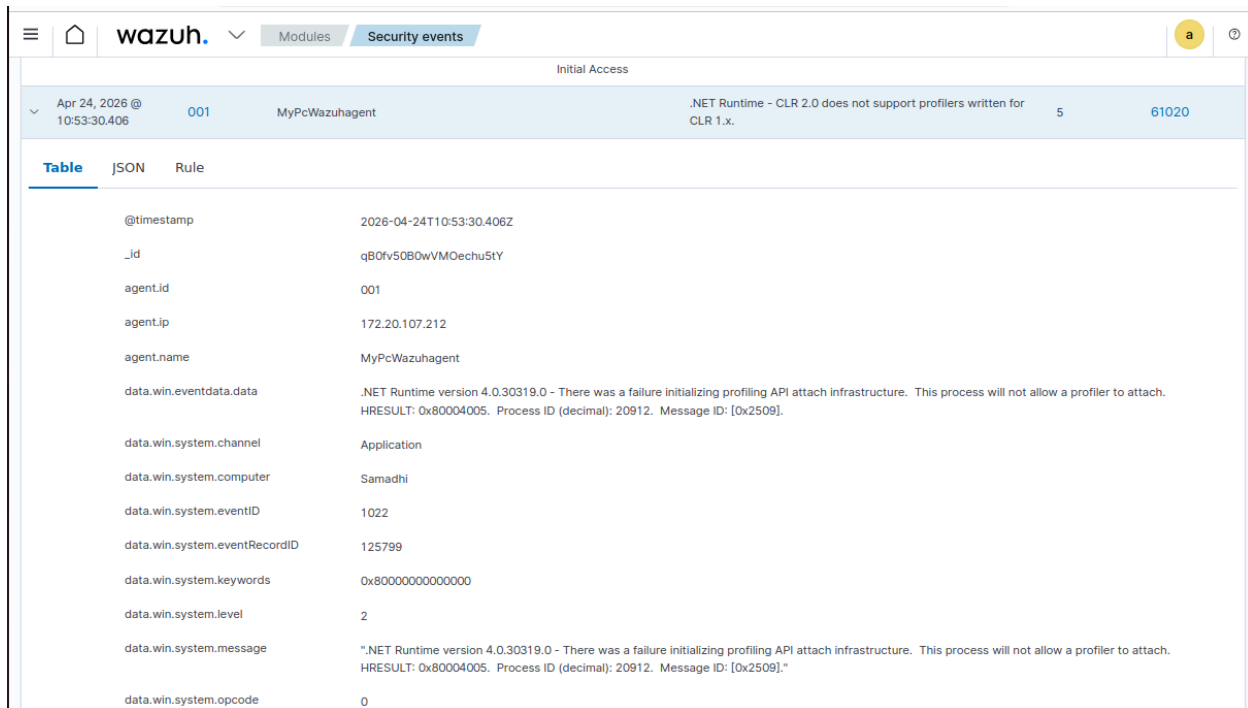
Every single alert carried the same signature:

- Techniques — T1078, T1531
- Tactics — Defense Evasion, Persistence, Privilege Escalation, Initial Access, Impact
- Description — Logon failure: unknown user or bad password
- Level — 5
- Rule ID — 60122

Five separate attempts. All within the same two-minute window between 15:18 and 15:59 on April 24, 2026.

This pattern indicates a potential brute-force attempt, which was successfully detected by Wazuh.

5.4 Application Logs Monitoring



The screenshot shows the Wazuh Security Events dashboard. At the top, there's a navigation bar with 'wazuh.' and 'Modules' and 'Security events' tabs. Below this, a header row indicates 'Initial Access'. The main content area displays a table of events. The first event is selected, showing details for 'Apr 24, 2026 @ 10:53:30.406', agent '001', and rule 'MyPcWazuagent'. The event message is '.NET Runtime - CLR 2.0 does not support profilers written for CLR 1.x.' with a severity of 5 and rule ID 61020. Below this, a 'Table' tab is active, showing a list of fields and their values for the selected event.

Field	Value
@timestamp	2026-04-24T10:53:30.406Z
_id	qB0fv50B0wVMOechu5tY
agent.id	001
agent.ip	172.20.107.212
agent.name	MyPcWazuagent
data.win.eventdata.data	.NET Runtime version 4.0.30319.0 - There was a failure initializing profiling API attach infrastructure. This process will not allow a profiler to attach. HRESULT: 0x80004005. Process ID (decimal): 20912. Message ID: [0x2509].
data.win.system.channel	Application
data.win.system.computer	Samadhi
data.win.system.eventID	1022
data.win.system.eventRecordID	125799
data.win.system.keywords	0x8000000000000000
data.win.system.level	2
data.win.system.message	".NET Runtime version 4.0.30319.0 - There was a failure initializing profiling API attach infrastructure. This process will not allow a profiler to attach. HRESULT: 0x80004005. Process ID (decimal): 20912. Message ID: [0x2509]."
data.win.system.opcode	0

Most application-level activity runs completely in the background errors firing, services struggling, runtime failures that never surface to the user. To bring all of that into view, we pointed Wazuh at the Windows Application Event channel by configuring the source inside `ossec.conf`.

Once that was in place, application events started coming through automatically. One alert stood out immediately.

At 10:53 AM on April 24, 2026, Wazuh flagged a .NET Runtime failure on agent MyPcWazuagent — tagged under Initial Access, Rule ID 61020, severity level 5.

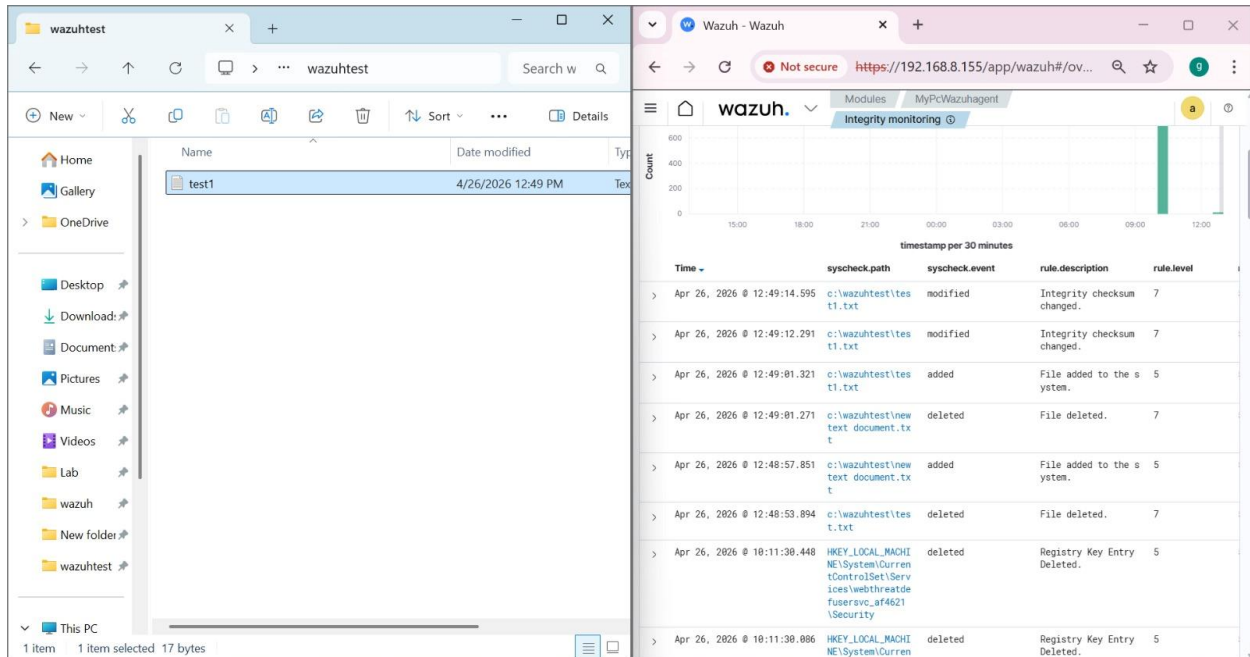
Expanding the event revealed exactly what happened:

- .NET Runtime 4.0.30319 failed to initialize its profiling API
- HRESULT: 0x80004005 — a general access failure code
- Process ID: 20912
- Message ID: 0x2509

The system logged into it. Wazuh caught it. And the dashboard surfaced it giving us a clear, time-stamped record of a runtime-level failure that would have otherwise gone completely unnoticed.

5.5 File Integrity Monitoring

File integrity monitoring was tested on a second Windows machine (IP: 192.168.8.146) connected to the same Wazuh server



Not every threat announces itself through login attempts or application errors. Sometimes an attacker leaves traces in the file system a file quietly added, modified, or deleted in the background [19].

To catch that, we enabled syscheck in ossec.conf and pointed it at a test directory: C:\wazuhtest. With real-time="yes" and check all="yes" active, Wazuh started watching every file change the moment it happened.

To test it, we created, modified and deleted files inside C:\wazuhtest and watched the dashboard respond in real time. Wazuh picked up every single change:

- File added to the system — Level 5
- Integrity checksum changed — Level 7
- File deleted — Level 7
- Registry Key Entry Deleted — Level 5

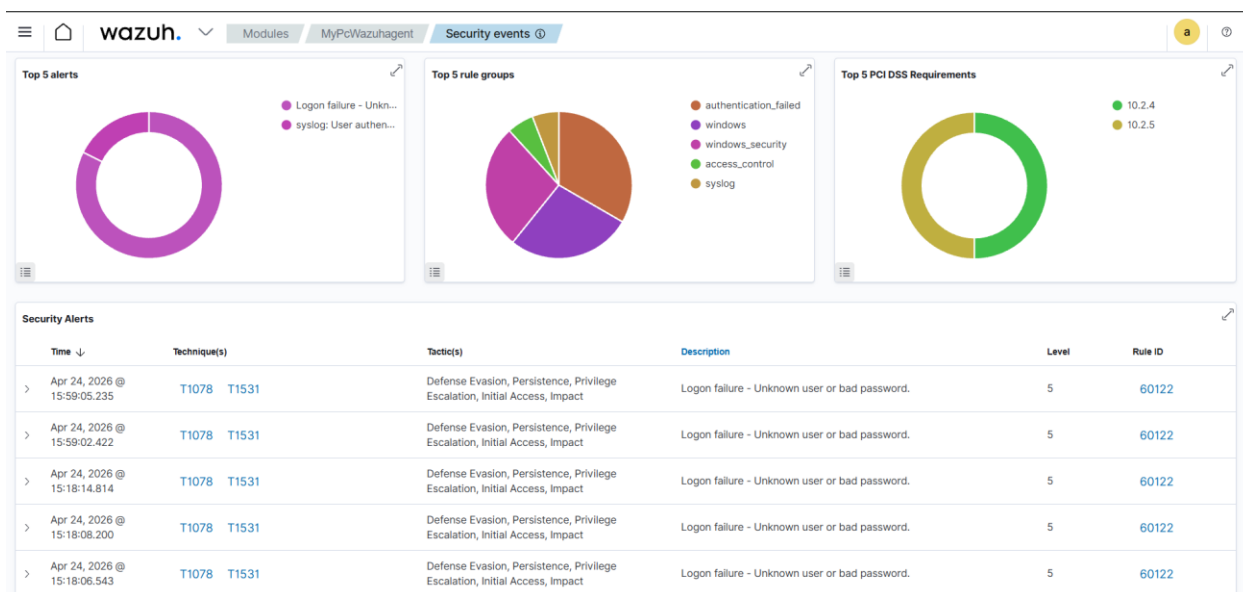
755 total integrity monitoring events were recorded across the session — giving a complete audit trail of exactly what changed, when, and where.

5.6 Demonstration of Simulated Events

To verify that the Wazuh system was functioning correctly, two security-related events were manually simulated on a Windows machine. The generated alerts were then observed through the Wazuh dashboard.

Scenario 1 — Multiple Failed Login Attempts

We entered wrong passwords repeatedly on the Windows machine. Each attempt immediately triggered an alert under Rule ID 60122 on the Wazuh dashboard — all five appearing in the Security Alerts table within the same timeframe.



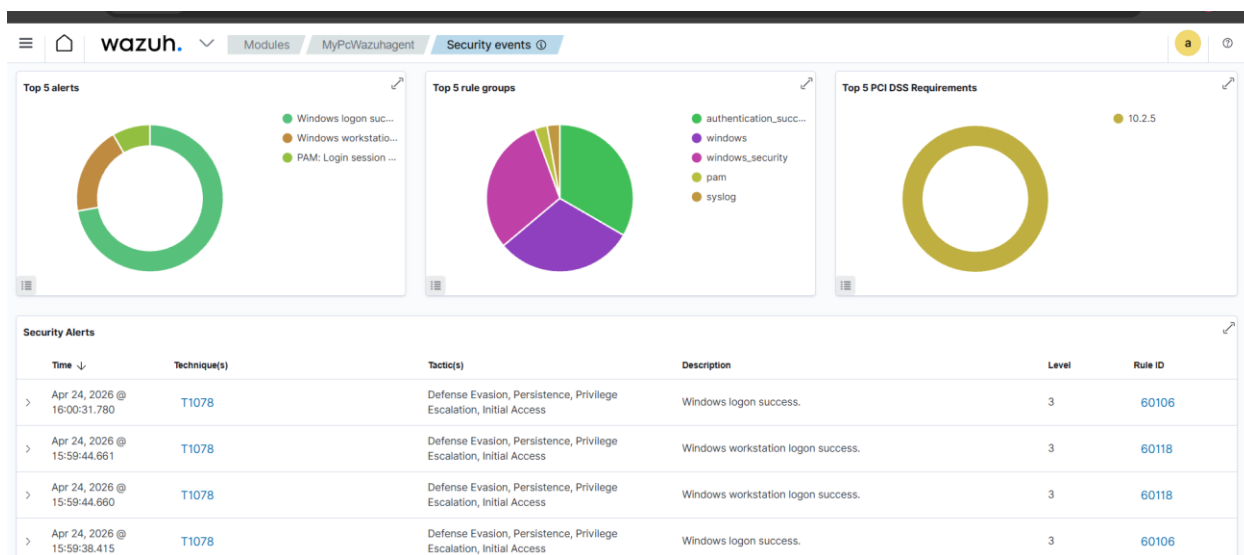
Every alert carried the same signature:

- Techniques — T1078, T1531
- Tactics — Defense Evasion, Persistence, Privilege Escalation, Initial Access, Impact
- Level — 5
- Description — Logon failure: unknown user or bad password

The clustering between 15:18 and 15:59 made the pattern obvious. Wazuh flagged every single attempt confirming the system can detect brute-force style login behaviour in real time.

Scenario 2— Privilege Escalation

Using a standard user account, we attempted to access administrator-level resources on the Windows machine. The system didn't just allow it quietly — Wazuh flagged it immediately.



The alerts appeared on the dashboard under the Security Alerts table, mapped to technique T1078 with tactics covering:

- Defense Evasion
- Persistence
- Privilege Escalation
- Initial Access

Two rule IDs fired:

- Rule 60106 — Windows logon success with elevated privileges
- Rule 60118 — Windows workstation logon success

All at Level 3, timestamped between 15:59 and 16:00 on April 24, 2026.

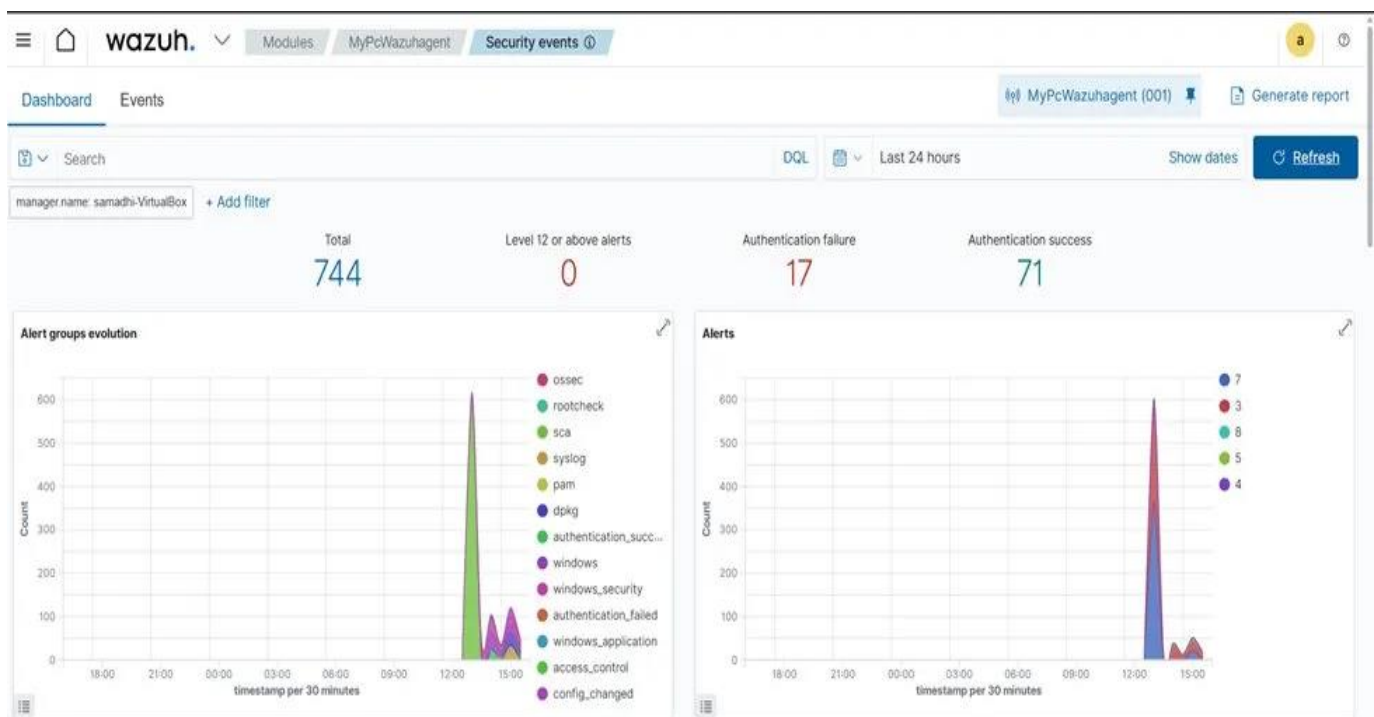
The Top 5 rule groups chart confirmed it — authentication_succ, windows and windows_security dominated the activity during this window.

This confirmed that any attempt to access privileged resources gets recorded, categorized and surfaced on the dashboard giving administrators a clear trail of who accessed what and when.

6. Screenshots of Implementation

The figures below show the dashboard during our monitoring session and the alerts that were generated from the simulated events [2].

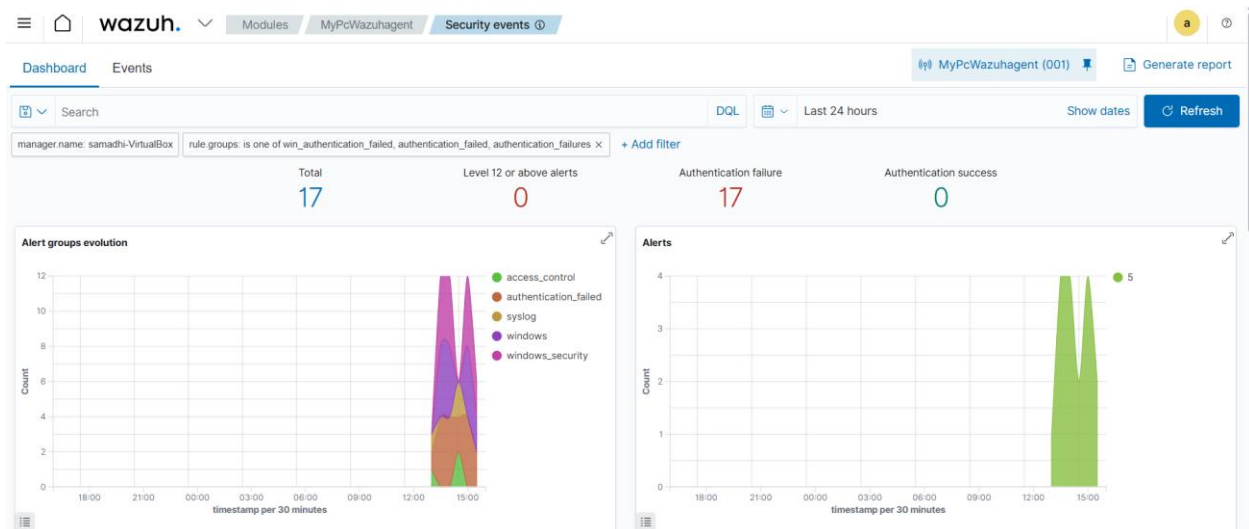
6.1 Wazuh Dashboard



The Security Events dashboard for agent MyPcWazuahagent recorded 744 total alerts over the monitoring session — including 17 authentication failures and 71 successes [2]. The alert spike visible around 15:00 reflects the simulated events triggered during our session.

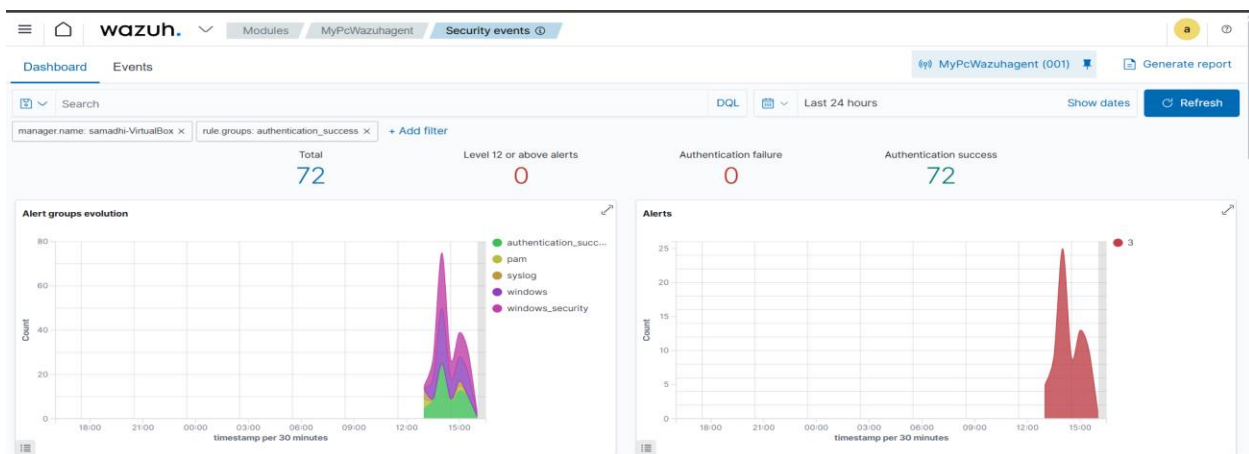
6.2 Security Events panel

Authentication failures



Authentication Failures in Wazuh Security Events Panel — 17 failed login attempts recorded, 0 successes. The Alert Groups Evolution chart highlights the spike in failure activity during the monitoring session.

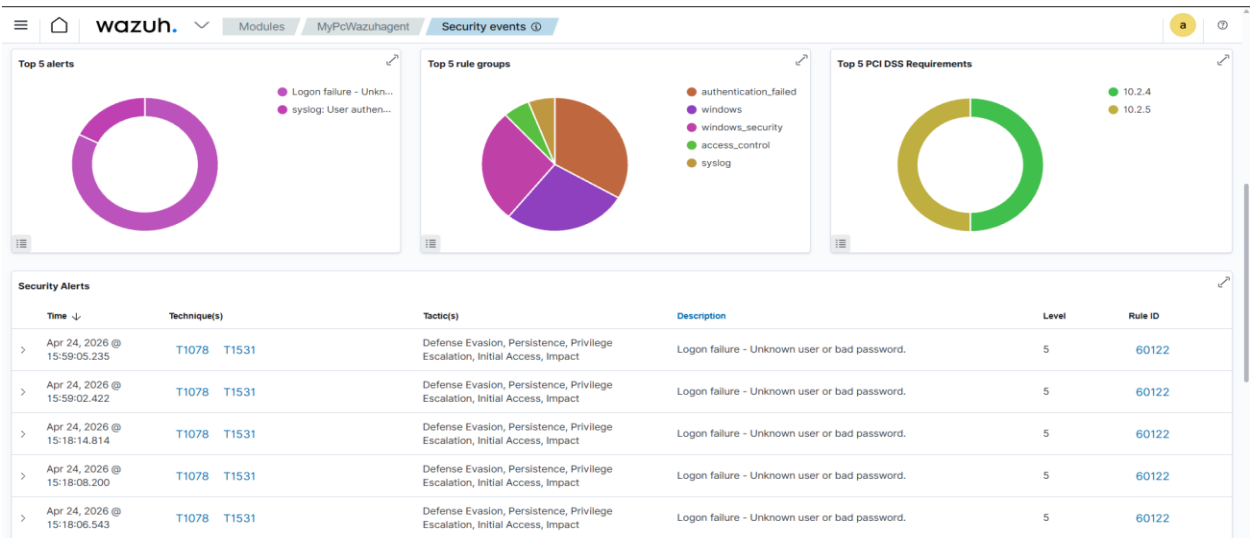
Authentication successes



Authentication Successes in Wazuh Security Events Panel — 72 successful logins recorded, 0 failures. The Rule Groups Evolution chart shows the spikes of successful login activity within the monitoring window.

6.3 Alert detail view

Security Alerts table with mixed events



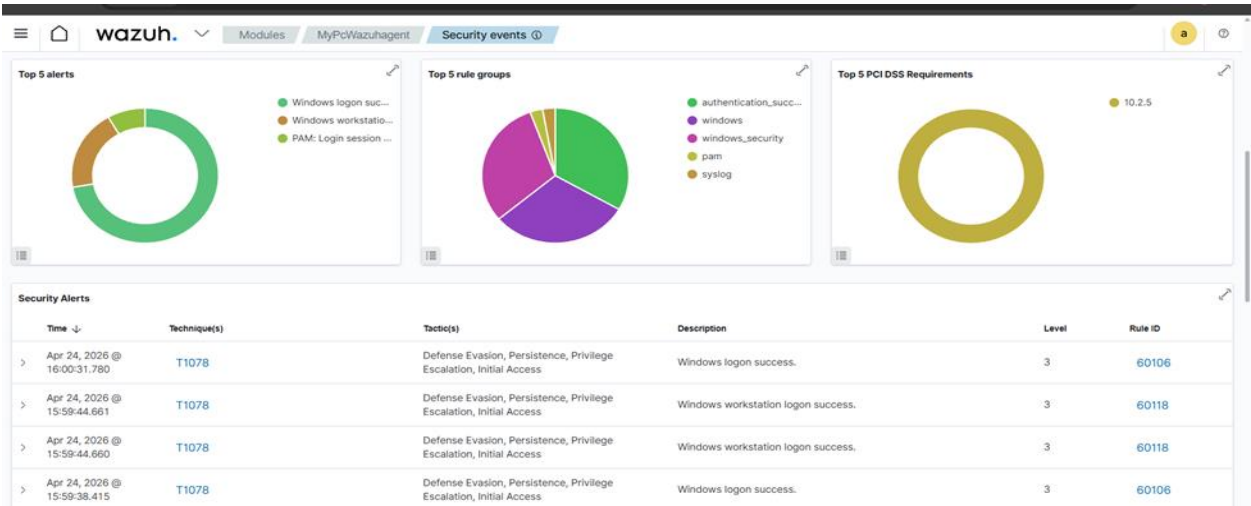
Alert detail showing rule ID, severity, and MITRE techniques (T1078, T1531).

The Security Alerts table displays individual alert entries with columns: Time, Technique, Tactics (Defense Evasion, Persistence, Privilege Escalation, Initial Access, Impact), Description (“Logon failure — Unknown user or bad password”), Level 5, and Rule ID 60122.

The Security Alerts table shows five logon failure events, all at:

Level 5 under Rule ID 60122, mapped to techniques T1078 and T1531

Windows logon success events



Alert detail view showing Windows logon success events under Rule IDs 60106 and 60118 with MITRE technique T1078.

The Security Alerts table includes successful login entries, descriptions “Windows logon success” and “Windows workstation logon success”.

An expanded alert detail view (not a table) shows full log entry fields including agent ID, IP address, decoder, and manager information for a system event alert.

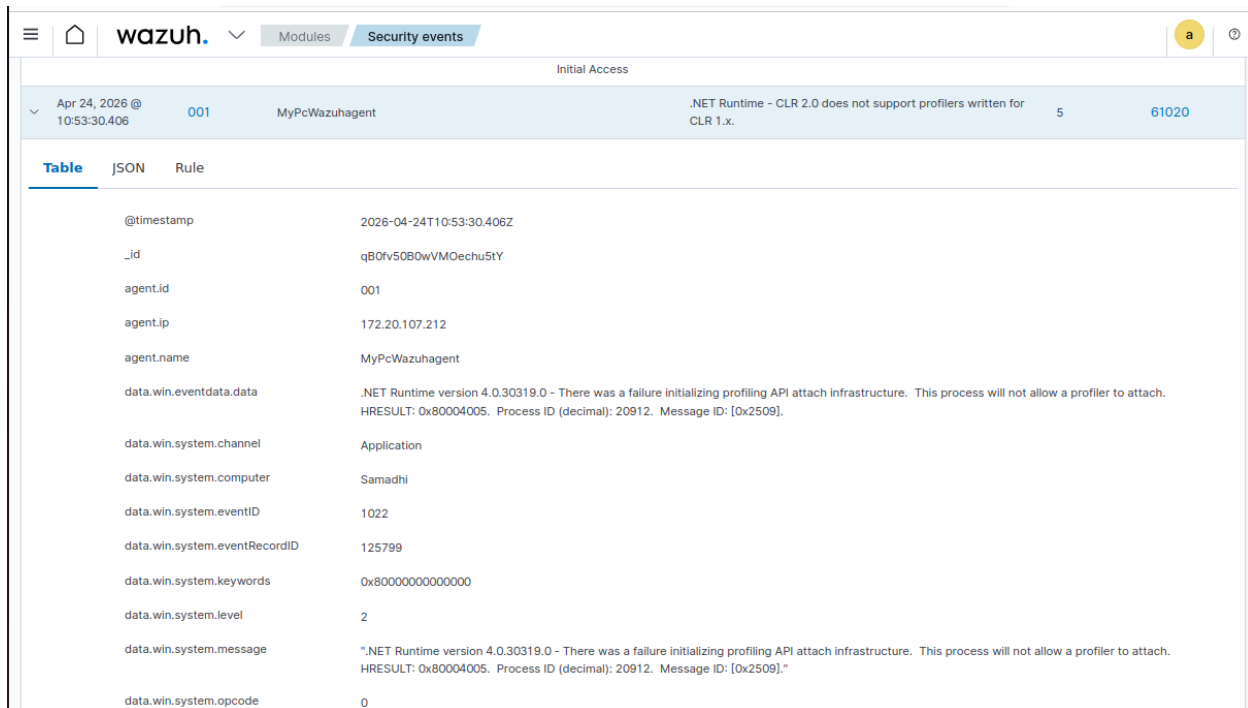
Agent start/stop events (non-Windows agent)

>	Apr 24, 2026 @ 15:43:18.312		Wazuh agent started.	3	503
~	Apr 24, 2026 @ 15:42:57.757	T1562.001	Defense Evasion	3	506

Table	JSON	Rule
	@timestamp	2026-04-24T10:12:57.757Z
	_id	ex36vp0B0wVMOeckpuv
	agent.id	001
 	agent.ip	172.20.107.212
	agent.name	MyPcWazuagent
	data.extra_data	MyPcWazuagent→any
	decoder.name	ossec
	decoder.parent	ossec
	full_log	ossec: Agent stopped: 'MyPcWazuagent→any'.
	id	1777025577.2753079
	input.type	log
	location	wazuh-remoted
	manager.name	samadhi-VirtualBox

So, this image shows a single alert in a JSON or table view. The agent's name is MxRcWazuhageot – this is not a Windows agent like the ones before. You can see the timestamp, agent.id 001, agent.ip, and agent.name MxRcWazuhageot. The decoder.name is ossec_full_log, and the log entry says "ossec: Agent stopped". Location is wazuh-remoted, and manager.name is samadhi-VirtualBox. The critical rule IDs here are 503 and 506 – those are for Wazuh-agent started and Wazuh-agent stopped. These events are urgent for agent health monitoring, but they have nothing to do with login success or failure.

6.4 NET Runtime alert



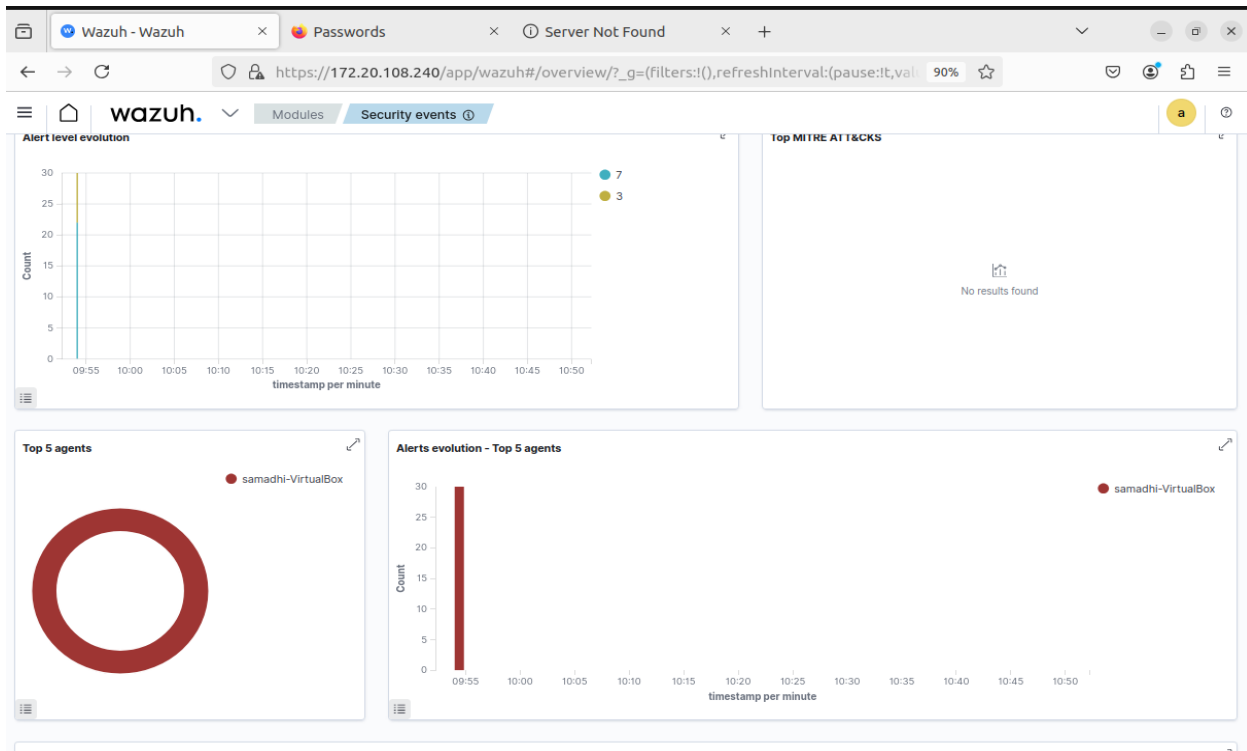
The screenshot shows the Wazuh Security Events interface. At the top, there's a navigation bar with 'wazuh.' and 'Modules' and 'Security events' tabs. Below this, a header row indicates 'Initial Access'. The main content area displays a table of security events. The first row shows an event from 'Apr 24, 2026 @ 10:53:30.406' with ID '001' from agent 'MyPcWazuahagent'. The event description is '.NET Runtime - CLR 2.0 does not support profilers written for CLR 1.x.' with a severity of '5' and a rule ID of '61020'. Below this, a detailed table lists the event's JSON data and the rule's configuration.

Table	JSON	Rule
@timestamp	2026-04-24T10:53:30.406Z	
_id	qB0fv50B0wVMOechu5tY	
agent.id	001	
agent.ip	172.20.107.212	
agent.name	MyPcWazuahagent	
data.win.eventdata.data	.NET Runtime version 4.0.30319.0 - There was a failure initializing profiling API attach infrastructure. This process will not allow a profiler to attach. HRESULT: 0x80004005. Process ID (decimal): 20912. Message ID: [0x2509].	
data.win.system.channel	Application	
data.win.system.computer	Samadhi	
data.win.system.eventID	1022	
data.win.system.eventRecordID	125799	
data.win.system.keywords	0x8000000000000000	
data.win.system.level	2	
data.win.system.message	.NET Runtime version 4.0.30319.0 - There was a failure initializing profiling API attach infrastructure. This process will not allow a profiler to attach. HRESULT: 0x80004005. Process ID (decimal): 20912. Message ID: [0x2509].	
data.win.system.opcode	0	

The image shows a .NET Runtime application alert detected on the Windows agent. The table displays agent.name MyPcWazuahagent, agent.id 001, and agent. ip172.20.107.212. Rule 61020 contains the description ".NET Runtime CLR 2.0 does not support profilers written for CLR 1.x", mapped to the Initial Access tactic at Level 5. The table includes data_win.system_name WINDOWS, system_id 10272, and duplicate entries for data_win.system_agentid. The accompanying text references system_eventID 1022 and computer Samadhi, though these fields are not visible in the table. This is a Level 5 .NET Runtime error under the Initial Access tactic.

6.5 MITRE ATT&CK Panel

Overview



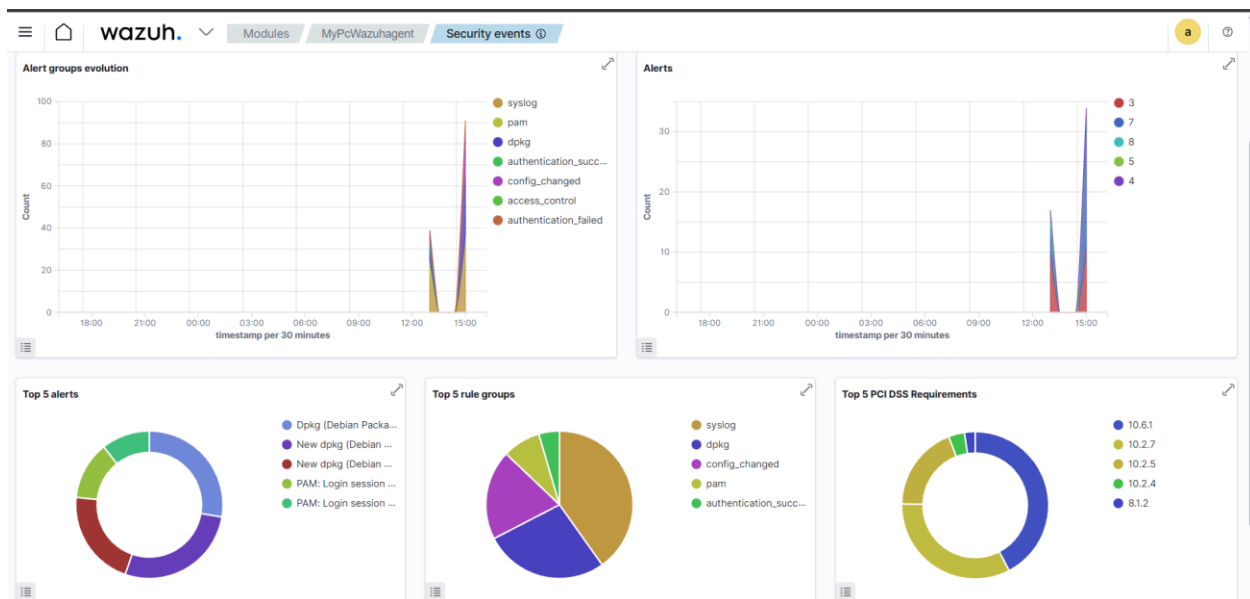
The Wazuh Security Events overview page includes a dedicated Top MITRE ATT&CK panel showing which attack techniques were detected during the session.

At the time this screenshot was captured, the panel displayed "No results found." This does not mean no techniques were detected — MITRE mapping was clearly visible at the individual alert level throughout the Security Events table:

- T1078 — mapped to all authentication events
- T1531 — mapped to logon failure alerts

The Alert level evolution chart showed level 7 and level 3 alerts active, and the Alerts Evolution chart confirmed a spike of activity around 09:55 with samadhi-VirtualBox as the active agent at that point. The MITRE panel reflects a broader time window and may not always match individual alert-level mappings in real time.

System Logs monitoring view



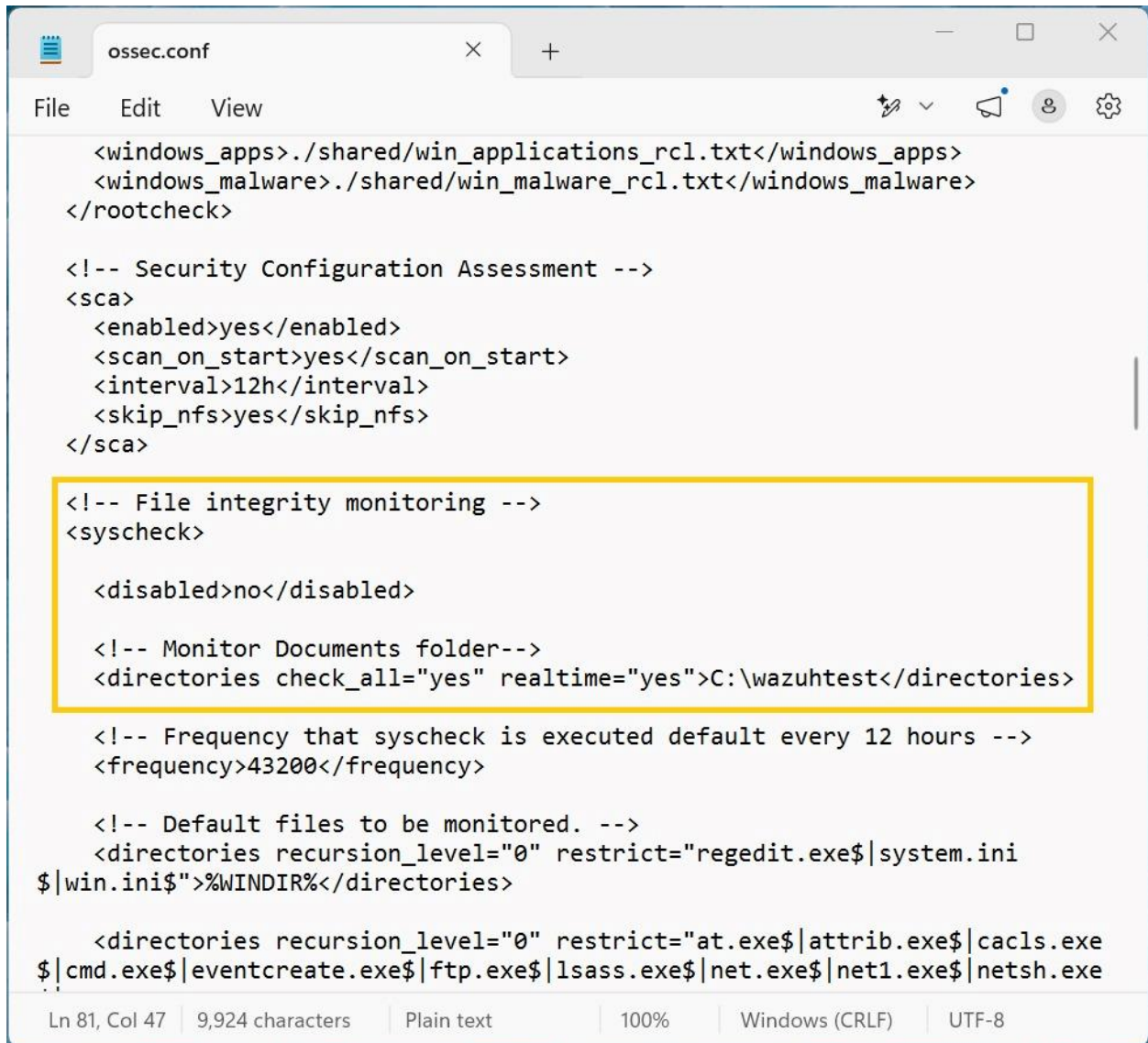
The System Logs monitoring view captured a broad range of activity across the session. The Alert Groups Evolution chart showed rule groups including syslog, pam, dpkg, authentication_succ, config_changed, access_control and authentication_failed all active during the monitoring window.

The Top 5 alerts donut displayed Dpkg Debian Package events and PAM login session events as the dominant activity. The Top 5 rule groups panel confirmed syslog, dpkg, config_changed, pam and authentication_succ as the most frequently triggered categories. PCI DSS compliance requirements flagged: 10.6.1, 10.2.7, 10.2.5, 10.2.4 and 8.1.2.

6.7 File Integrity Monitoring Dashboard

Note: Screenshots in this section were captured from a second Windows PC (IP: 192.168.8.146). Both machines used the same Wazuh server at 192.168.8.155.

FIM Configuration



```
<windows_apps>./shared/win_applications_rcl.txt</windows_apps>
<windows_malware>./shared/win_malware_rcl.txt</windows_malware>
</rootcheck>

<!-- Security Configuration Assessment -->
<sca>
  <enabled>yes</enabled>
  <scan_on_start>yes</scan_on_start>
  <interval>12h</interval>
  <skip_nfs>yes</skip_nfs>
</sca>

<!-- File integrity monitoring -->
<syscheck>

  <disabled>no</disabled>

  <!-- Monitor Documents folder-->
  <directories check_all="yes" realtime="yes">C:\wazuhtest</directories>

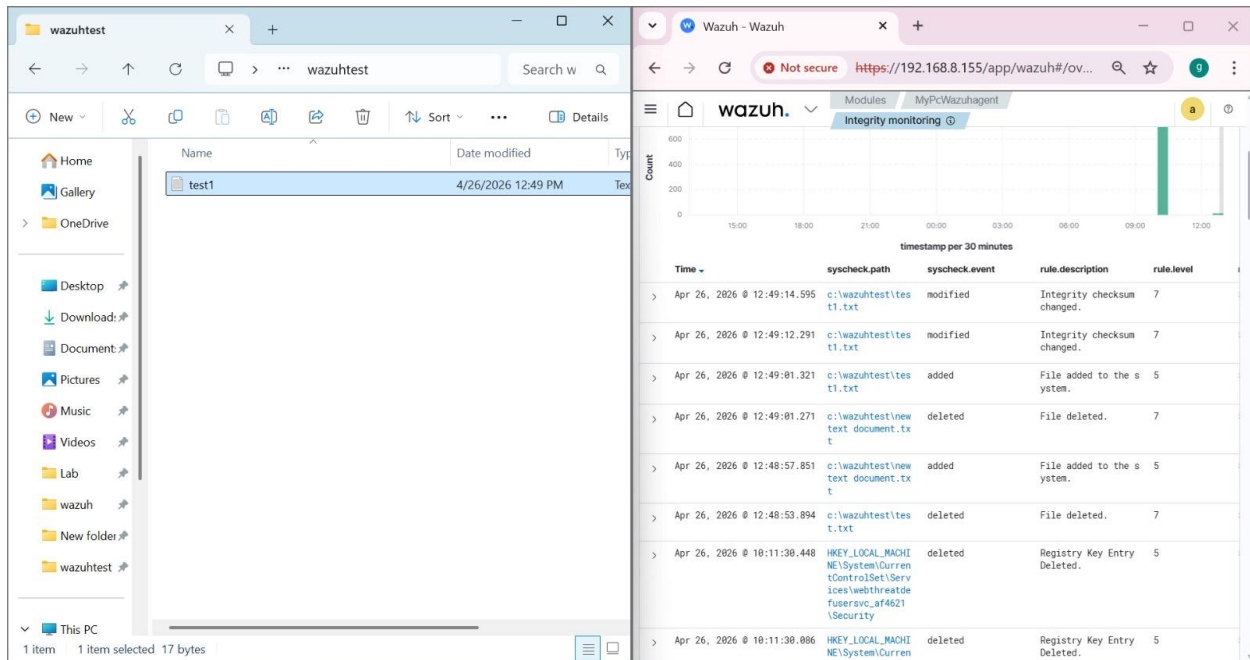
  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <!-- Default files to be monitored. -->
  <directories recursion_level="0" restrict="regedit.exe$|system.ini
$|win.ini$">%WINDIR%</directories>

  <directories recursion_level="0" restrict="at.exe$|attrib.exe$|cacls.exe
$|cmd.exe$|eventcreate.exe$|ftp.exe$|lsass.exe$|net.exe$|net1.exe$|netsh.exe
$|
```

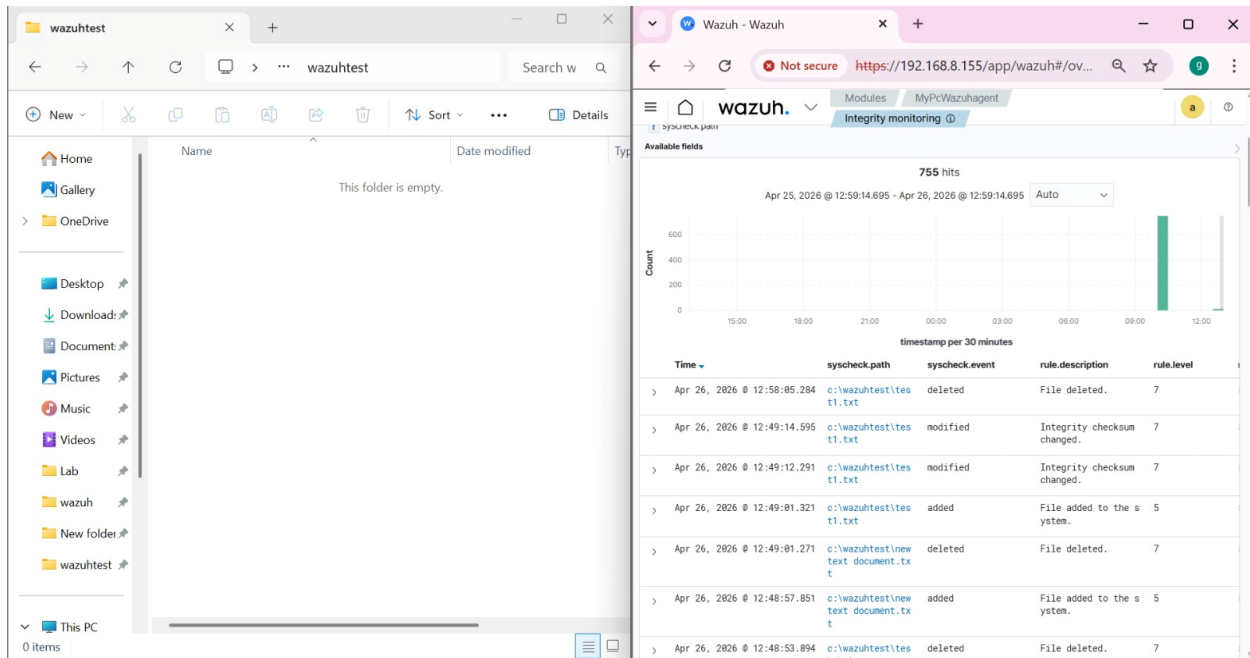
The ossec.conf syscheck configuration confirms file integrity monitoring was actively enabled on the C:\wazuhtest directory with real-time detection and full checksum verification.

File Events — Active Monitoring



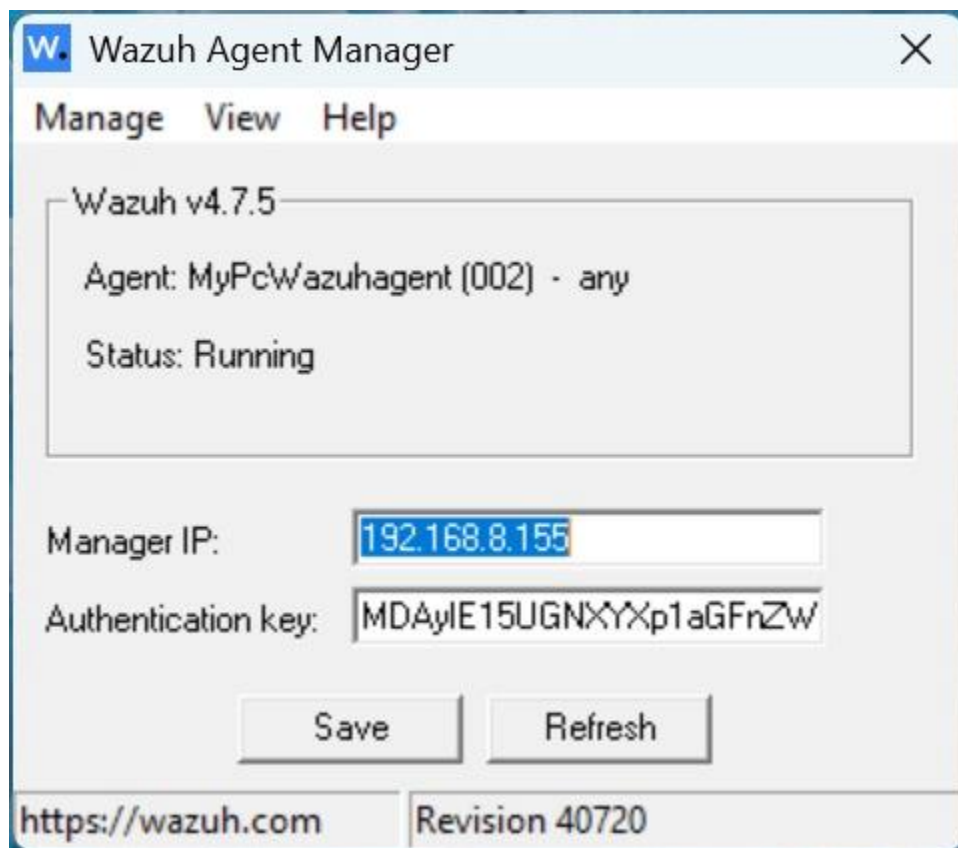
With test1.txt present in the wazuhtest folder, the Integrity Monitoring dashboard captured all file activity in real time. Key events recorded:

File Events — Folder Cleared



After all files were removed from the wazuhtest folder, the dashboard confirmed 755 total integrity monitoring hits across the session. The folder on the left shows completely empty while the dashboard on the right continued logging every event in real time, confirming nothing was missing.

Agent Status Confirmation



The Wazuh Agent Manager confirmed the agent was actively connected throughout the session:

- Agent — MyPcWazuhagent (002)
- Status — Running
- Manager IP — 192.168.8.155
- Wazuh Version — v4.7.5

7. Security Event Detection and Analysis

7.1 Overview of Detected Security Events

Various security alerts were generated by Wazuh throughout the monitoring session. The dashboard gave alerts such as logon failure, successful login, NET Runtime alert, and start/stop alerts for agents. The alert gave information such as agent name, rule ID, level, timestamp, and MITRE ATT&CK techniques were relevant. On the MITRE ATT&CK page on the overview, it read, "No results found."

7.2 Summary of Triggered Alerts

The table below shows the main alerts that Wazuh detected during our session:

Alert	Agent	Rule ID	Level	MITRE Technique
Logon failure — unknown user or bad password	MyPcWazuhagent	60122	5	T1078, T1531
Windows workstation logon success	MyPcWazuhagent	60118	3	T1078
.NET Runtime error	MyPcWazuhagent	61020	5	Initial Access
Agent stopped	MyPcWazuhagent	506	3	None
Agent started	MyPcWazuhagent	503	3	None
File added to the system	MyPcWazuhagent	554	5	None
Integrity checksum changed	MyPcWazuhagent	550	7	None
File deleted	MyPcWazuhagent	553	7	None
Registry Key Entry Deleted	MyPcWazuhagent	750	5	None

The Top 5 rule groups observed were: syslog, dpkg, config_changed, pam, and authentication success. The Top 5 PCI DSS requirements observed were:

10.6.1, 10.2.7, 10.2.5, 10.2.4, and 8.1.2.

8. Analysis of Detected Events

8.1 Detailed Alert Investigation

The following alerts were investigated based on the session

Alert 1 – Logon failure (Rule ID: 60122)

The detection of this alert involved a Windows agent that worked at the Level 5 alert level. Technique mapping for this alert included MITRE T1078 (Valid Accounts) and T1531 (Account Access Removal) [12],[13]. Tactics associated with these techniques were defense evasion, persistence, privilege escalation, initial access, and impact. Some logon failures occurred at roughly the same timestamp of April 24, 2026, at 15:19:08.

Alert 2 – .NET Runtime error (Rule ID 61020)

This alert was triggered on agent MyPcWazuagent (agent ID 001, IP172.20.107.212) at Level 5. It mapped to the Initial Access tactic [5]. The description stated that ".NET Runtime CLR 2.0 does not support profilers written for CLR 1.x". System details showed Windows as the system name, system ID 10272, and computer name Samadhi.

Alert #3 – Agent start/stop events (Rule IDs 503 and 506)

These events occurred at agent MyPcWazuagent. (agent ID 001). The decoder was ossec_full_log, location was wazuh-remoted, and manager was samadhi-VirtualBox. The log entry read "ossec: Agent stopped", indicating these were operational events for agent health monitoring rather than security threats [3].

Alert #4 — File Integrity Monitoring Events

File integrity alerts were triggered on agent MyPcWazuagent across the C:\wazuhtest directory on April 26, 2026 [19]. Four types of events were detected:

- File added — Level 5 — a new file appeared in the monitored directory
- Integrity checksum changed — Level 7 — an existing file was modified

- File deleted — Level 7 — a file was removed from the monitored directory
- Registry Key Entry Deleted — Level 5 — a Windows registry entry was removed

A total of 755 integrity monitoring events were recorded — each with full file path, timestamp, event type and rule level, providing a complete audit trail of all changes made inside the monitored directory [19].

8.2 Root Cause and Attack Methodology

The failed login alerts were caused by multiple attempts using incorrect passwords, creating the same pattern as a brute-force attack [10]. Brute force means an attacker tries to guess a password repeatedly, sometimes using automated scripts [14]. Wazuh detected each attempt individually and flagged the clustering as suspicious activity exactly the kind of early warning a real security team would act on.

The file integrity monitoring events were caused by deliberate file operations inside the C:\wazuhtest directory creating, modifying and deleting files to simulate how an attacker might plant or remove files on a compromised system [19]. Wazuh detected every change in real time through the syscheck engine, confirming that file-level tampering cannot go unnoticed on a monitored endpoint.

8.3 Threat Impact Assessment

If these events had occurred on a real system, the consequences could be serious. A successful brute-force attack could give an attacker full access to a user account and everything that account can reach [14]. Getting admin privileges would give the attacker control over the entire machine — including the ability to disable security tools or steal data. Wazuh detecting all events quickly would give a real security team enough time to respond and stop the attack before serious damage was done.

8.4 How Wazuh Identifies and Responds to Threats

Wazuh works by collecting log data from agents and sending it to the server. The server decodes the raw log into structured fields — username, event type, source IP — then checks the event against its detection rules [3]. When a match is found, it creates an alert with a severity level. When the same event occurs repeatedly in a short window, Wazuh recognises the pattern and raises the alert priority [10]. All alerts are saved and shown on the dashboard, each mapped to a MITRE ATT&CK category, so analysts immediately understand what technique was used and what the attacker's likely goal was [6].

9. Conclusion

9.1 Summary of Findings

We set up the Wazuh SIEM platform in a virtual lab environment, installing the server on an Ubuntu machine and connecting a Windows agent. Log collection was configured on the Windows agent for system, authentication and application logs. Several attack scenarios were then simulated including failed login attempts, privilege escalation, and file integrity violations on the Windows machine and Wazuh caught every single one through the dashboard. That confirmed what we set out to prove: Wazuh works as a capable security monitoring tool even in a basic lab setup.

9.2 Reflection on Wazuh's Effectiveness

Wazuh proved more useful than we initially expected. Every simulated attack surfaced on the dashboard without any custom rule writing the built-in detection handled it. What stood out most was the MITRE ATT&CK mapping. Instead of just logging into an event, Wazuh immediately told us what technique was being used and what the attacker's likely goal was. In a real company environment that kind of context matters especially when monitoring multiple machines at the same time.

9.3 Possible Improvements

A few things could make the setup stronger. Enabling active response would allow Wazuh to automatically block suspicious IPs when a brute-force attempt is detected rather than just alerting. Turning the alert rules to reduce unnecessary low-level noise would also help, making it easier to focus on what matters. Email notifications for high-severity alerts would be a practical addition too so the team stays informed even when no one is actively watching the dashboard.

9.4 Future Work

There's room to expand this significantly. Adding firewall logs and web server logs would give much broader coverage. Writing custom detection rules for specific threat types would sharpen accuracy. Connecting Wazuh to a threat intelligence feed would let it automatically flag known malicious IPs as they appear. And deploying it in a cloud environment would be a natural next step to see how well it handles monitoring cloud-based infrastructure at scale.

10. References

- [2] Wazuh, Inc., "What is Wazuh?," *Wazuh Documentation*, 2024. [Online]. Available: <https://documentation.wazuh.com/current/getting-started/index.html>. [Accessed: Apr. 2025].
- [3] Wazuh, Inc., "Wazuh Installation Guide — All-in-one Deployment," *Wazuh Documentation*, 2024. [Online]. Available: <https://documentation.wazuh.com/current/installation-guide/wazuh-server/index.html>. [Accessed: Apr. 2025].
- [4] Wazuh, Inc., "Wazuh Agent Deployment on Windows," *Wazuh Documentation*, 2024. [Online]. Available: <https://documentation.wazuh.com/current/installation-guide/wazuh-agent/wazuh-agent-package-windows.html>. [Accessed: Apr. 2025].
- [5] Wazuh, Inc., "Log Data Analysis," *Wazuh Documentation*, 2024. [Online]. Available: <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/index.html>. [Accessed: Apr. 2025].
- [6] MITRE Corporation, "MITRE ATT&CK Framework," 2024. [Online]. Available: <https://attack.mitre.org>. [Accessed: Apr. 2025].
- [7] Center for Internet Security, "CIS Microsoft Windows 11 Benchmark," *CIS Benchmarks*, 2024. [Online]. Available: <https://www.cisecurity.org/cis-benchmarks>. [Accessed: Apr. 2025].
- [8] A. Chuvakin, K. Schmidt, and C. Phillips, *Logging and Log Management: The Authoritative Guide to Understanding the Concepts Surrounding Logging and Log Management*, 1st ed. Waltham, MA: Syngress, 2013.
- [9] Wazuh, Inc., "Monitoring Windows Event Log," *Wazuh Documentation*, Wazuh, Inc., Austin, TX, USA, 2024. [Online]. Available: <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/monitoring-windows-event-log.html>. [Accessed: Apr. 10, 2025].
- [10] Wazuh, Inc., "Detecting Brute Force Attacks," *Wazuh Proof of Concept Guide*, Wazuh, Inc., Austin, TX, USA, 2024. [Online]. Available: <https://documentation.wazuh.com/current/proof-of-concept-guide/detect-brute-force-attack.html>. [Accessed: Apr. 10, 2025].
- [11] Wazuh, Inc., "Active Response," *Wazuh Documentation*, Wazuh, Inc., Austin, TX, USA, 2024. [Online]. Available: <https://documentation.wazuh.com/current/user-manual/capabilities/active-response/index.html>. [Accessed: Apr. 10, 2025].
- [12] MITRE Corporation, "T1078: Valid Accounts," *MITRE ATT&CK Enterprise Matrix*, MITRE Corporation, McLean, VA, USA, 2024. [Online]. Available: <https://attack.mitre.org/techniques/T1078>. [Accessed: Apr. 10, 2025].

[13] MITRE Corporation, "T1531: Account Access Removal," *MITRE ATT&CK Enterprise Matrix*, MITRE Corporation, McLean, VA, USA, 2024. [Online]. Available: <https://attack.mitre.org/techniques/T1531>. [Accessed: Apr. 10, 2025].

[14] MITRE Corporation, "T1110: Brute Force," *MITRE ATT&CK Enterprise Matrix*, MITRE Corporation, McLean, VA, USA, 2024. [Online]. Available: <https://attack.mitre.org/techniques/T1110>. [Accessed: Apr. 10, 2025].

[15] Microsoft Corporation, "Event 4625 — An Account Failed to Log On," *Microsoft Security Auditing Documentation*, Microsoft Corporation, Redmond, WA, USA, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4625>. [Accessed: Apr. 10, 2025].

[16] Microsoft Corporation, "Event 4624 — An Account Was Successfully Logged On," *Microsoft Security Auditing Documentation*, Microsoft Corporation, Redmond, WA, USA, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4624>. [Accessed: Apr. 10, 2025].

[17] Microsoft Corporation, "Event 4672 — Special Privileges Assigned to New Logon," *Microsoft Security Auditing Documentation*, Microsoft Corporation, Redmond, WA, USA, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4672>. [Accessed: Apr. 10, 2025].

[18] Payment Card Industry Security Standards Council, "Payment Card Industry Data Security Standard (PCI DSS) v4.0," *PCI SSC Document Library*, PCI Security Standards Council, Wakefield, MA, USA, Mar. 2022. [Online]. Available: https://www.pcisecuritystandards.org/document_library. [Accessed: Apr. 10, 2025].

[19] Wazuh, Inc., "File Integrity Monitoring," *Wazuh Documentation*, Wazuh, Inc., Austin, TX, USA, 2024. [Online]. Available: <https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/index.html>. [Accessed: Apr. 26, 2025].